

[Day 7 종합실습] JOIN · 서브쿼리 · 윈도우함수 복합 쿼리 실습

AI 서비스를 위한 SW 기초 Full-stack Engineering — 스마트 데이터 이해 및 활용 | 종합실습-2 | 광주캠퍼스 4반 박영서 | 2026-07-23

실습 결과 요약

- 실행환경: PostgreSQL 17.10 — DB skala_db · 스키마 lab · 사용자 skala_user (일반 권한 롤)
- 데이터: student 1,000 · enroll 2,302(고아 수강 2건 포함) · customers 500 · orders 3,000 · emp 311
- 문항 1~25 전부 실제 실행 — 문항별로 ① psql CLI + ② pgAdmin 4 Query Tool 양쪽 실행 화면(쿼리+결과 동시 표시) 수록
- 교재 지시(Postgres/MySQL: LIMIT 5)에 따라 대량 결과는 LIMIT 5로 출력 제한, 주석에 전체 행 수 명시(Q8·Q13은 문항 지정 건수 유지)
- 채점 기준 준수 — 요구사항 1:1 반영(문항 원문을 쿼리 주석으로 표기), SELECT * 미사용(필요 컬럼만 선택)
- 쿼리 2개 문항(12·22·25)은 쿼리별로 각각 실행·캡처, Q23은 서브쿼리·CTE 두 방식 모두 작성

0. 실습 환경 확인

```
parkyoungseo — psql -h localhost -U skala_user -d skala_db — 99x31
skala_db-> \i env.sql
\conninfo
접속 정보: 데이터베이스="skala_db", 사용자="skala_user", 호스트="localhost" (주소="::1"), 포트="5432"
.
\dt lab.*
      릴레이션 목록
스키마 | 이름      | 형태 | 소유주
-----+-----+-----+-----
lab    | course_owner | 테이블 | skala_user
lab    | customers   | 테이블 | parkyoungseo
lab    | emp         | 테이블 | parkyoungseo
lab    | enroll      | 테이블 | parkyoungseo
lab    | orders      | 테이블 | parkyoungseo
lab    | student     | 테이블 | parkyoungseo
(6개 행)

SELECT 'student' AS t, COUNT(*) FROM student
UNION ALL SELECT 'enroll', COUNT(*) FROM enroll
UNION ALL SELECT 'customers', COUNT(*) FROM customers
UNION ALL SELECT 'orders', COUNT(*) FROM orders
UNION ALL SELECT 'emp', COUNT(*) FROM emp;
 t      | count
-----+-----
student | 1000
enroll  | 2302
customers | 500
orders  | 3000
emp      | 311
(5개 행)

skala_db-> 
```

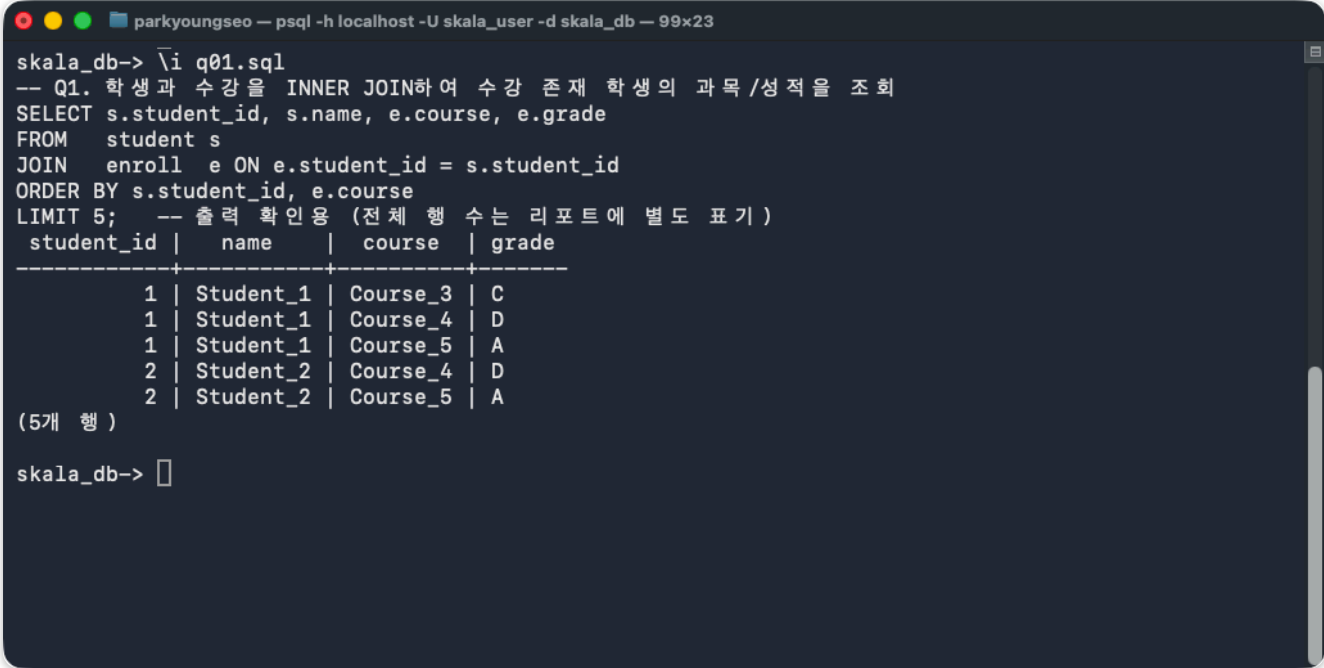
skala_user로 skala_db 접속(\conninfo), lab 스키마 테이블과 테이블별 건수 확인.

1. 학생과 수강을 INNER JOIN하여 수강 존재 학생의 과목/성적을 조회

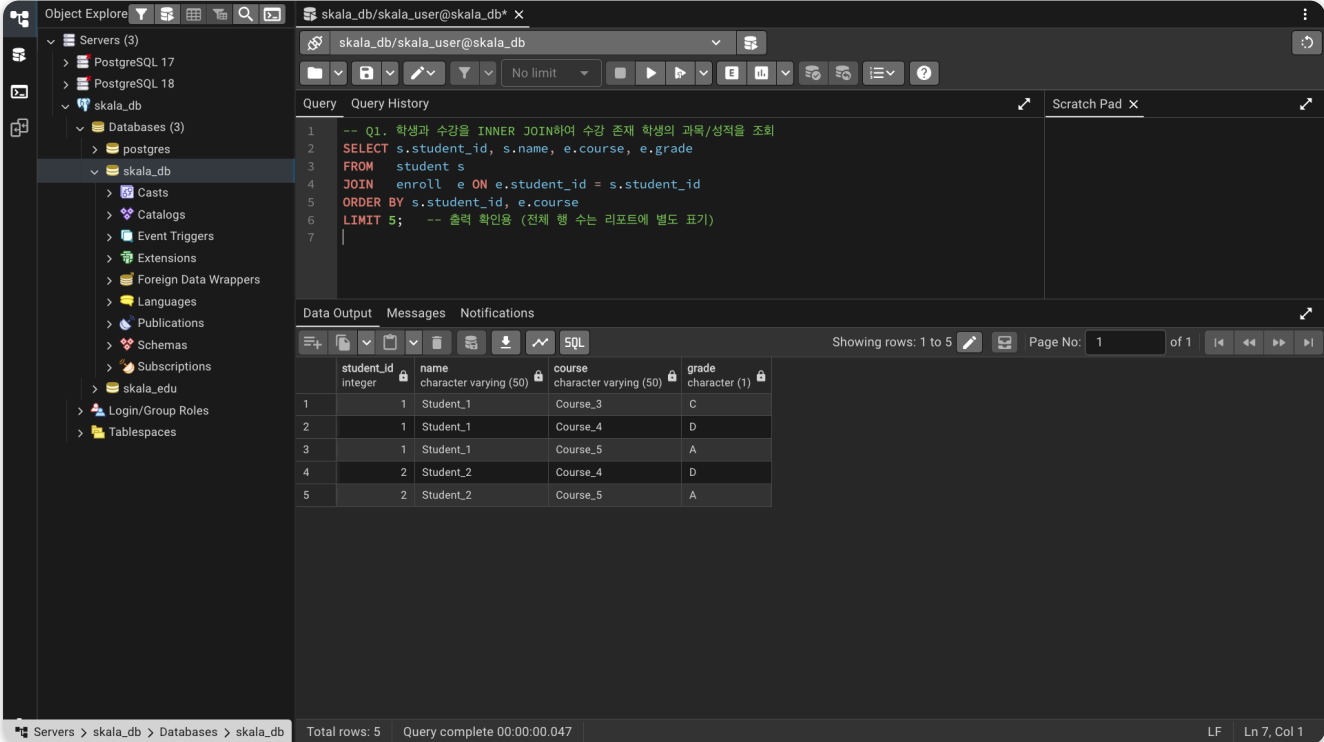
SQL

```
-- Q1. 학생과 수강을 INNER JOIN하여 수강 존재 학생의 과목/성적을 조회
SELECT s.student_id, s.name, e.course, e.grade
FROM   student s
JOIN   enroll e ON e.student_id = s.student_id
ORDER BY s.student_id, e.course
LIMIT 5;  -- 출력 확인용 (전체 행 수는 리포트에 별도 표기)
```

실행 결과 ① — psql



실행 결과 ② — pgAdmin 4 Query Tool



INNER JOIN — 양쪽 매칭 행만 조회 (전체 2,300행 중 상위 5행 표시).

2. 모든 학생 기준으로 수강을 붙이고, 과목(없으면 NULL)까지 보이기

SQL

```
-- Q2. 모든 학생 기준으로 수강을 붙이고, 과목(없으면 NULL)까지 보이기 (LEFT JOIN)
SELECT s.student_id, s.name, e.course
FROM   student s
LEFT JOIN enroll e ON e.student_id = s.student_id
ORDER BY s.student_id, e.course
LIMIT 5;  -- 출력 확인용 (전체 행 수는 리포트에 별도 표기)
```

실행 결과 ① — psql



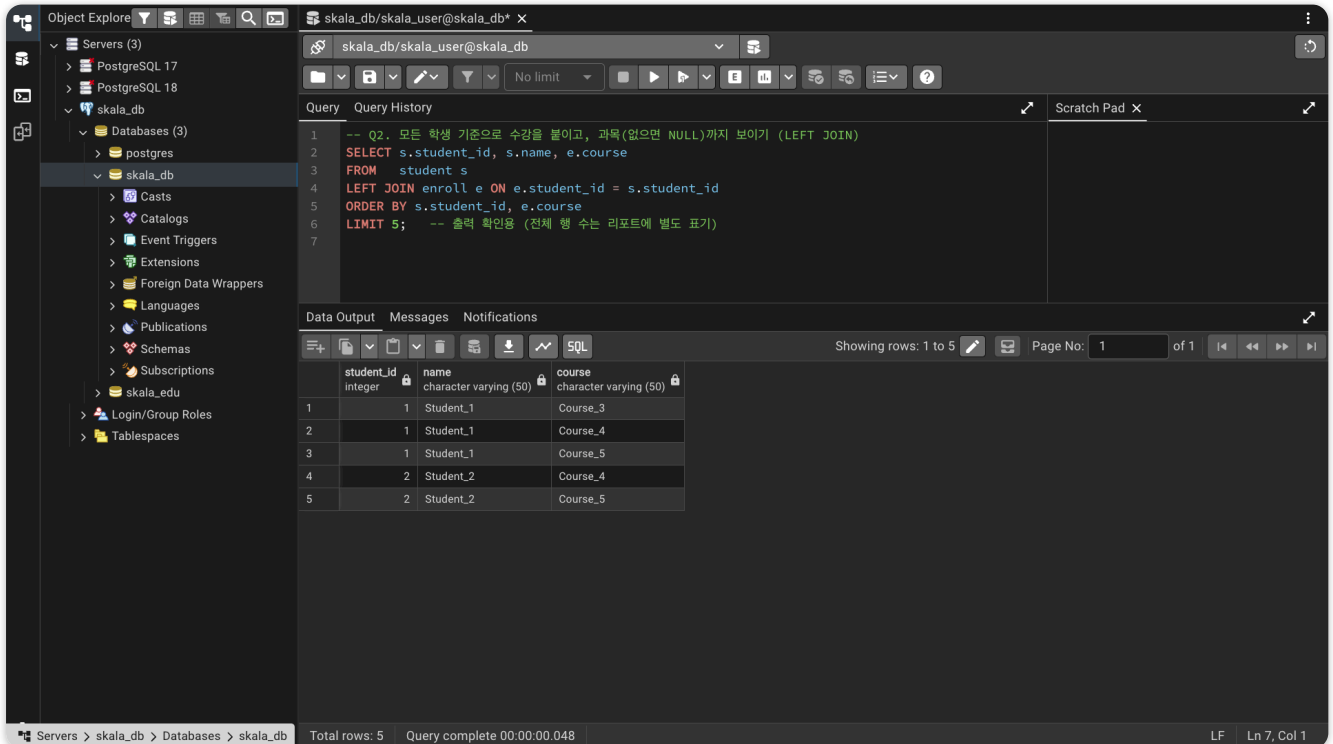
```
parkyoungseo — psql -h localhost -U skala_user -d skala_db — 99x23

skala_db-> \i q02.sql
-- Q2. 모든 학생 기준으로 수강을 붙이고, 과목(없으면 NULL)까지 보이기 (LEFT JOIN)
SELECT s.student_id, s.name, e.course
FROM   student s
LEFT JOIN enroll e ON e.student_id = s.student_id
ORDER BY s.student_id, e.course
LIMIT 5;  -- 출력 확인용 (전체 행 수는 리포트에 별도 표기)
 student_id |  name  | course
-----+-----+-----
          1 | Student_1 | Course_3
          1 | Student_1 | Course_4
          1 | Student_1 | Course_5
          2 | Student_2 | Course_4
          2 | Student_2 | Course_5

(5개 행 )

skala_db-> 
```

실행 결과 ② — pgAdmin 4 Query Tool



The screenshot shows the pgAdmin 4 Query Tool interface. The query editor contains the same SQL query as in the psql terminal. The Data Output pane shows the results of the query, which are 5 rows of data.

student_id	name	course
1	Student_1	Course_3
1	Student_1	Course_4
1	Student_1	Course_5
2	Student_2	Course_4
2	Student_2	Course_5

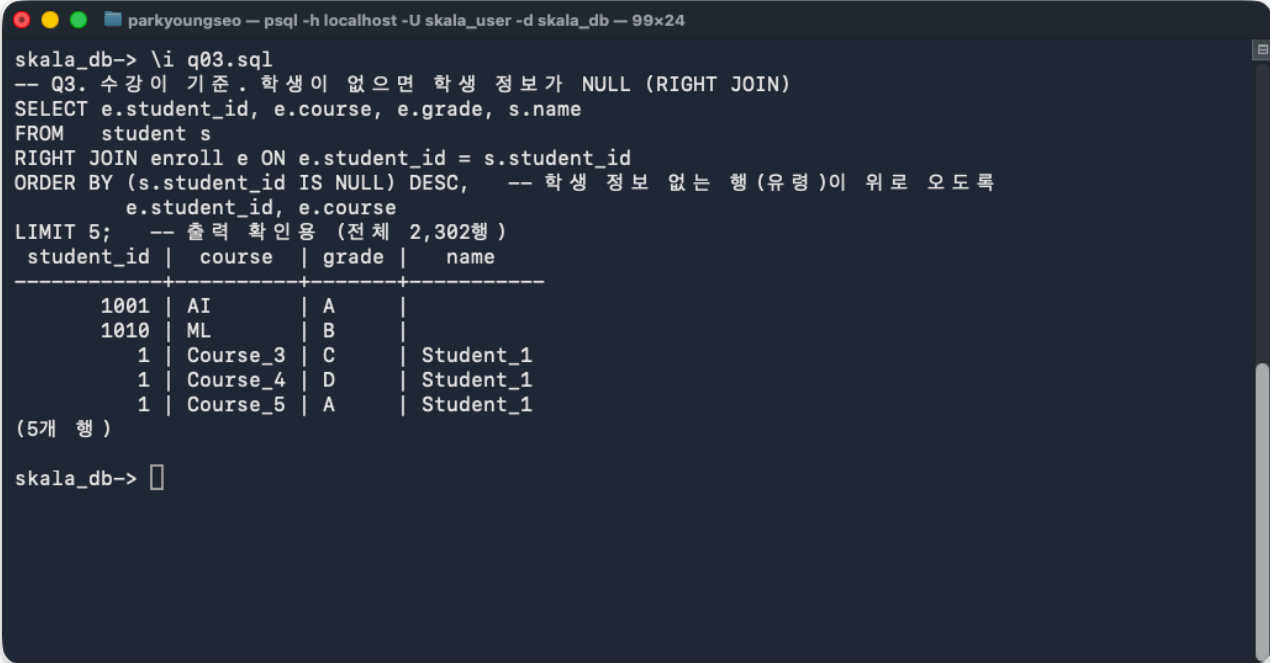
LEFT JOIN — 미수강 학생은 course가 NULL로 표시 (전체 2,400행).

3. 수강이 기준. 학생이 없으면 학생 정보가 NULL (RIGHT JOIN)

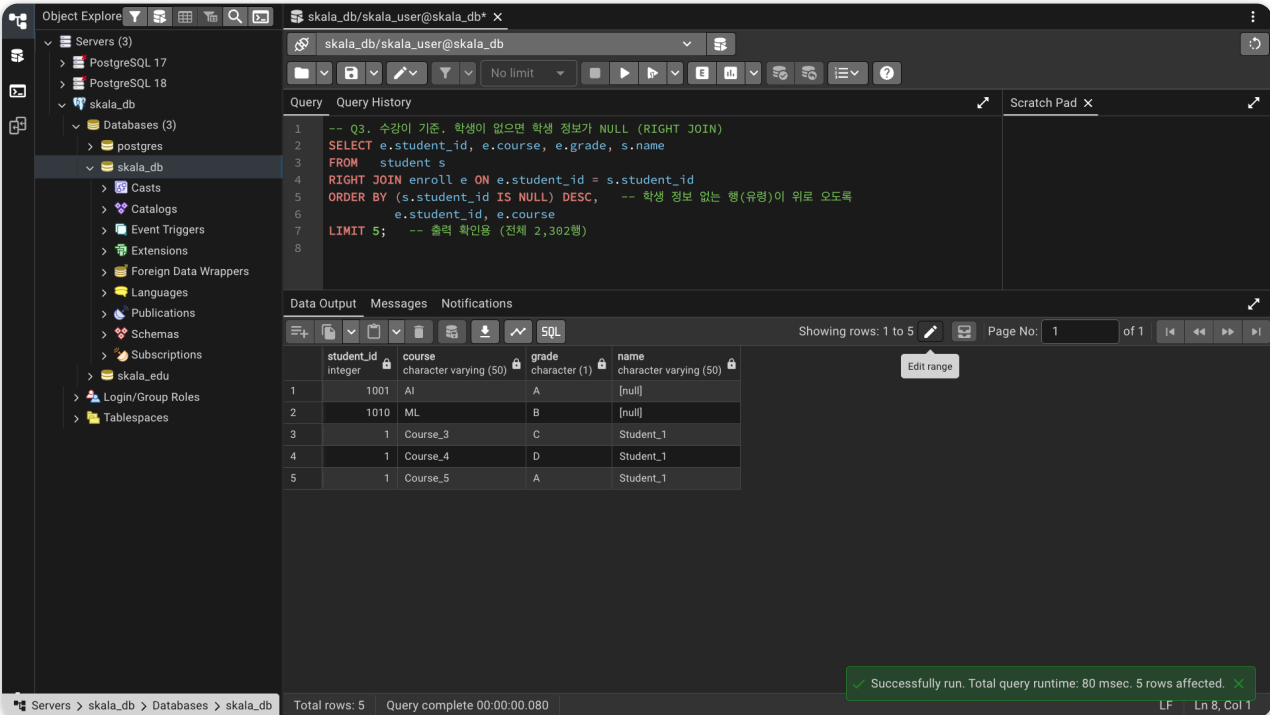
SQL

```
-- Q3. 수강이 기준. 학생이 없으면 학생 정보가 NULL (RIGHT JOIN)
SELECT e.student_id, e.course, e.grade, s.name
FROM   student s
RIGHT JOIN enroll e ON e.student_id = s.student_id
ORDER BY (s.student_id IS NULL) DESC,  -- 학생 정보 없는 행(유령)이 위로 오도록
        e.student_id, e.course
LIMIT 5;  -- 출력 확인용 (전체 2,302행)
```

실행 결과 ① — psql



실행 결과 ② — pgAdmin 4 Query Tool



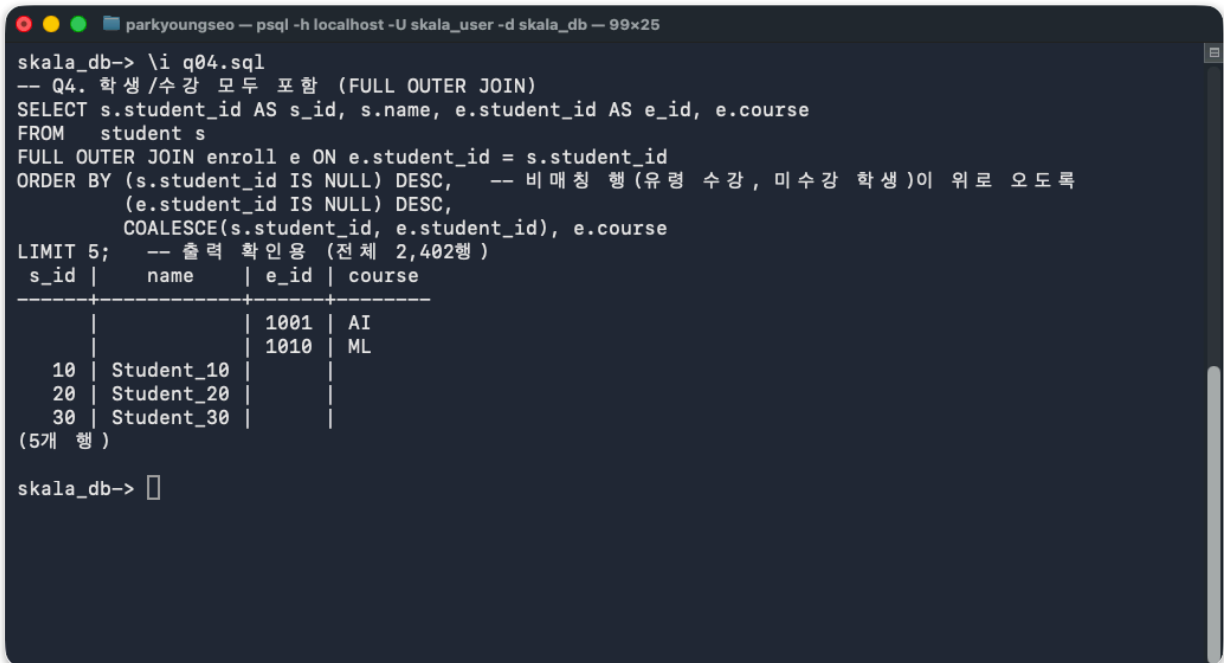
RIGHT JOIN — 학생 정보가 없는 고아 수강(1001·1010)이 name NULL로 상단 표시 (전체 2,302행).

4. 학생/수강 모두 포함 (FULL OUTER JOIN)

SQL

```
-- Q4. 학생/수강 모두 포함 (FULL OUTER JOIN)
SELECT s.student_id AS s_id, s.name, e.student_id AS e_id, e.course
FROM   student s
FULL OUTER JOIN enroll e ON e.student_id = s.student_id
ORDER BY (s.student_id IS NULL) DESC, -- 비매칭 행(유령 수강, 미수강 학생)이 위로 오도록
         (e.student_id IS NULL) DESC,
         COALESCE(s.student_id, e.student_id), e.course
LIMIT 5; -- 출력 확인용 (전체 2,402행)
```

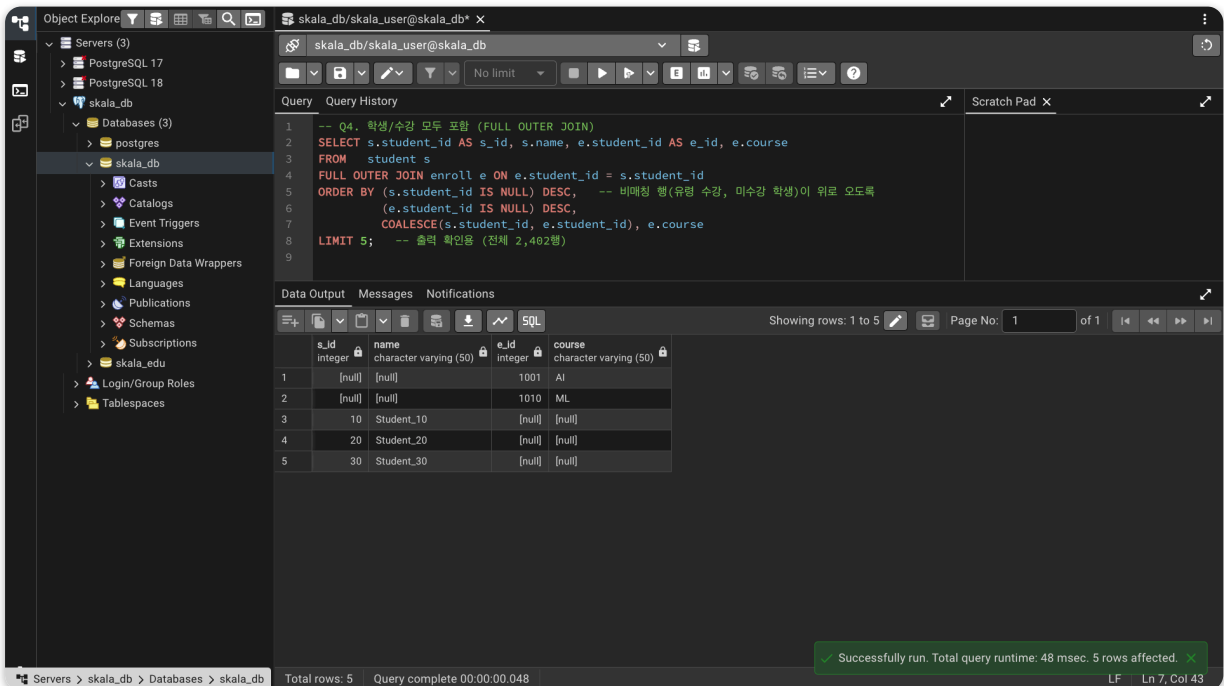
실행 결과 ① — psql



```
scala_db-> \i q04.sql
-- Q4. 학생/수강 모두 포함 (FULL OUTER JOIN)
SELECT s.student_id AS s_id, s.name, e.student_id AS e_id, e.course
FROM   student s
FULL OUTER JOIN enroll e ON e.student_id = s.student_id
ORDER BY (s.student_id IS NULL) DESC, -- 비매칭 행(유령 수강, 미수강 학생)이 위로 오도록
         (e.student_id IS NULL) DESC,
         COALESCE(s.student_id, e.student_id), e.course
LIMIT 5; -- 출력 확인용 (전체 2,402행)
 s_id |      name      | e_id | course
-----+-----+-----+-----
      |               |      | 
10    | Student_10    |      | 
20    | Student_20    |      | 
30    | Student_30    |      | 
(5개 행)

scala_db-> 
```

실행 결과 ② — pgAdmin 4 Query Tool



The screenshot shows the pgAdmin 4 Query Tool interface. The query is executed, and the results are displayed in the Data Output tab. The output table has 5 rows and 4 columns: s_id, name, e_id, and course. The first two rows show non-matching rows from the enroll table (e_id is null), and the next three rows show matching rows from the student table.

s_id	name	e_id	course
[null]	[null]	1001	AI
[null]	[null]	1010	ML
10	Student_10	[null]	[null]
20	Student_20	[null]	[null]
30	Student_30	[null]	[null]

Successfully run. Total query runtime: 48 msec. 5 rows affected.

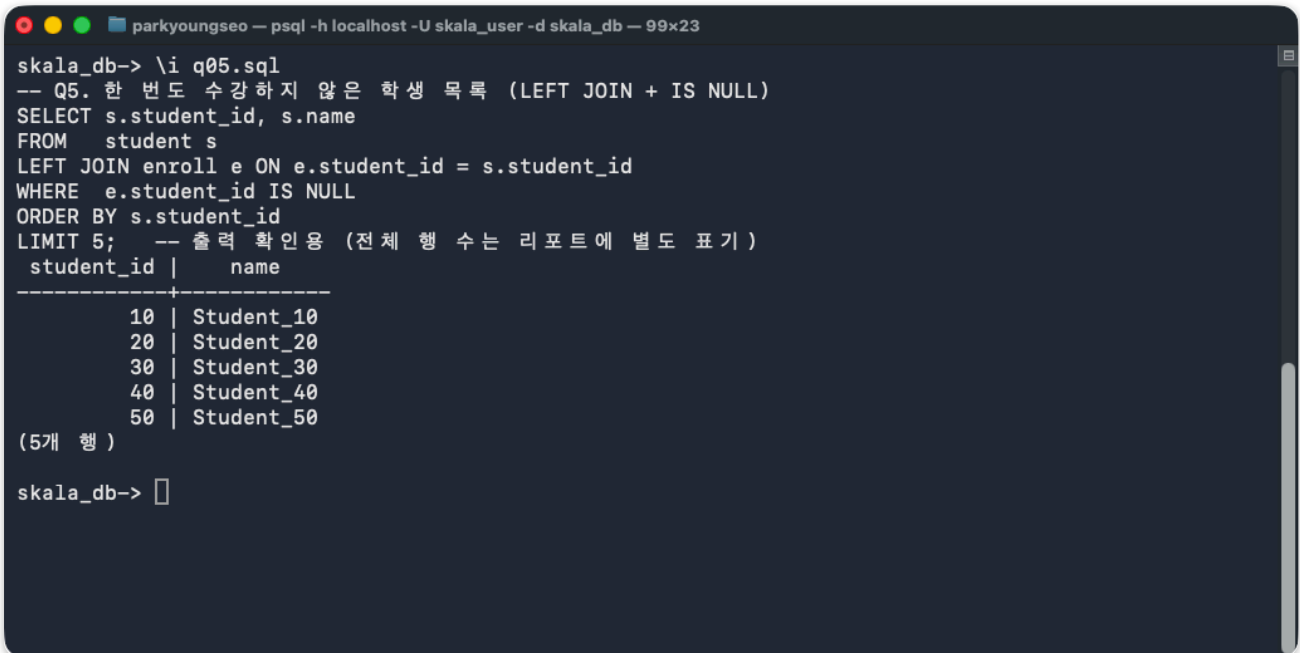
FULL OUTER JOIN — 고아 수강 2건 + 미수강 학생이 함께 포함 (전체 2,402행).

5. 한 번도 수강하지 않은 학생 목록

SQL

```
-- Q5. 한 번도 수강하지 않은 학생 목록 (LEFT JOIN + IS NULL)
SELECT s.student_id, s.name
FROM   student s
LEFT JOIN enroll e ON e.student_id = s.student_id
WHERE  e.student_id IS NULL
ORDER BY s.student_id
LIMIT 5;  -- 출력 확인용 (전체 행 수는 리포트에 별도 표기)
```

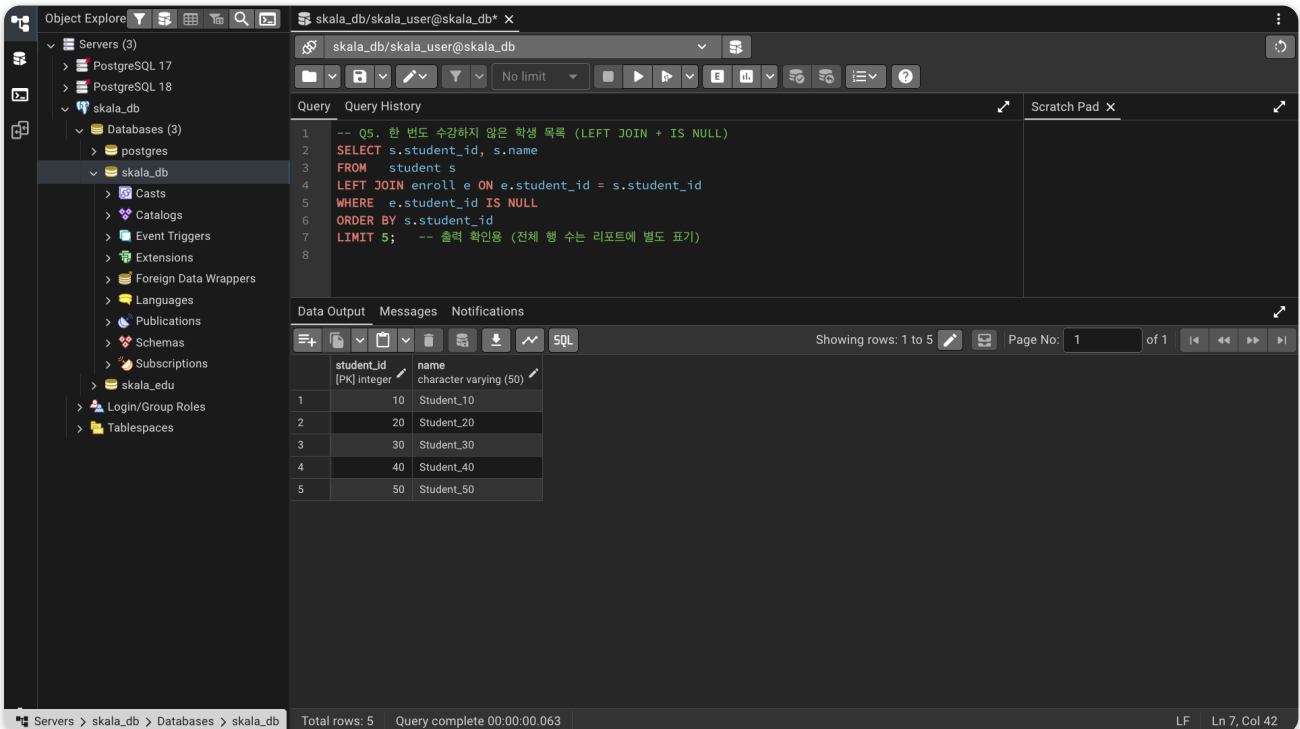
실행 결과 ① — psql



```
skala_db-> \i q05.sql
-- Q5. 한 번도 수강하지 않은 학생 목록 (LEFT JOIN + IS NULL)
SELECT s.student_id, s.name
FROM   student s
LEFT JOIN enroll e ON e.student_id = s.student_id
WHERE  e.student_id IS NULL
ORDER BY s.student_id
LIMIT 5;  -- 출력 확인용 (전체 행 수는 리포트에 별도 표기)
 student_id | name
-----+-----
          10 | Student_10
          20 | Student_20
          30 | Student_30
          40 | Student_40
          50 | Student_50
(5개 행)

skala_db-> []
```

실행 결과 ② — pgAdmin 4 Query Tool



```
1 -- Q5. 한 번도 수강하지 않은 학생 목록 (LEFT JOIN + IS NULL)
2 SELECT s.student_id, s.name
3 FROM   student s
4 LEFT JOIN enroll e ON e.student_id = s.student_id
5 WHERE  e.student_id IS NULL
6 ORDER BY s.student_id
7 LIMIT 5;  -- 출력 확인용 (전체 행 수는 리포트에 별도 표기)
8
```

student_id [PK] integer	name character varying (50)
1	10 Student_10
2	20 Student_20
3	30 Student_30
4	40 Student_40
5	50 Student_50

Total rows: 5 Query complete 00:00:00.063

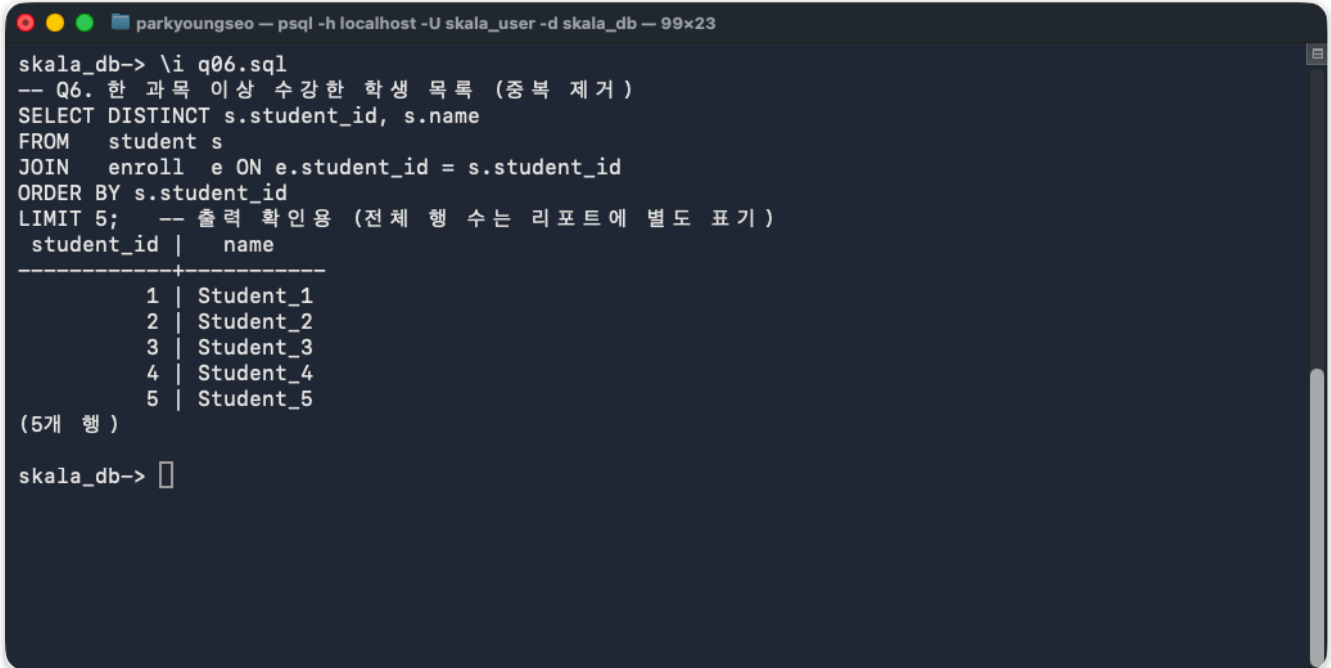
LEFT JOIN + IS NULL 안티 조인 — 미수강 학생 전체 100명 중 상위 5명.

6. 한 과목 이상 수강한 학생 목록 (중복 제거)

SQL

```
-- Q6. 한 과목 이상 수강한 학생 목록 (중복 제거)
SELECT DISTINCT s.student_id, s.name
FROM   student s
JOIN   enroll e ON e.student_id = s.student_id
ORDER BY s.student_id
LIMIT 5;  -- 출력 확인용 (전체 행 수는 리포트에 별도 표기)
```

실행 결과 ① — psql

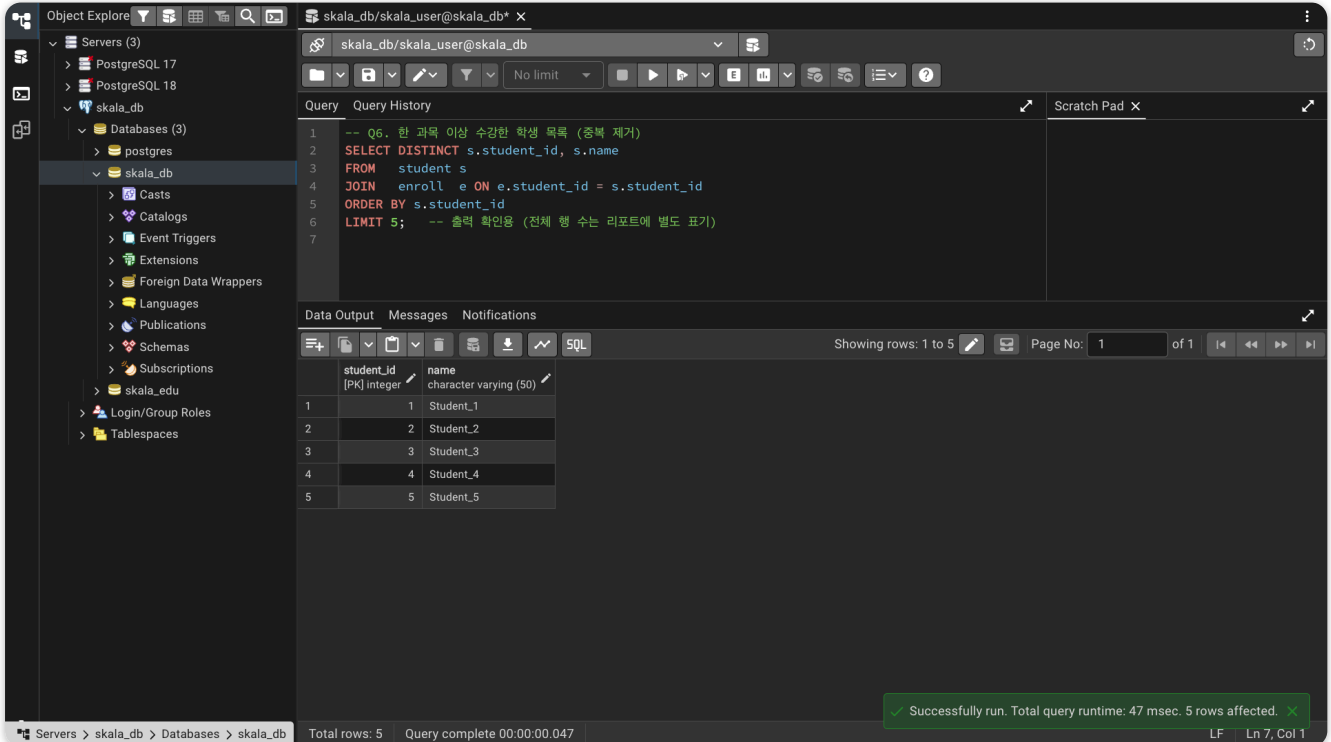


```
parkyoungseo — psql -h localhost -U skala_user -d skala_db — 99x23

skala_db-> \i q06.sql
-- Q6. 한 과목 이상 수강한 학생 목록 (중복 제거)
SELECT DISTINCT s.student_id, s.name
FROM   student s
JOIN   enroll e ON e.student_id = s.student_id
ORDER BY s.student_id
LIMIT 5;  -- 출력 확인용 (전체 행 수는 리포트에 별도 표기)
 student_id |  name
-----+-----
          1 | Student_1
          2 | Student_2
          3 | Student_3
          4 | Student_4
          5 | Student_5
(5개 행)

skala_db-> 
```

실행 결과 ② — pgAdmin 4 Query Tool



The screenshot shows the pgAdmin 4 Query Tool interface. The left sidebar displays the database structure, including the 'skala_db' database. The main query editor contains the same SQL query as shown in the psql terminal. The 'Data Output' tab is active, displaying the results of the query in a table format. The table has two columns: 'student_id' (integer) and 'name' (character varying (50)). The results show 5 rows of data, corresponding to the students enrolled in the course.

student_id	name
1	Student_1
2	Student_2
3	Student_3
4	Student_4
5	Student_5

Successfully run. Total query runtime: 47 msec. 5 rows affected.

JOIN + DISTINCT 세미 조인 — 수강 이력 보유 학생 900명 중 상위 5명.

7. 고객별 주문건수/총액

SQL

```
-- Q7. 고객별 주문건수/총액
SELECT c.customer_id, c.customer_name, COUNT(*) AS order_count, SUM(o.amount) AS total_amount
FROM   customers c
JOIN   orders o ON o.customer_id = c.customer_id
GROUP BY c.customer_id, c.customer_name
ORDER BY c.customer_id
LIMIT 5;  -- 출력 확인용 (전체 행 수는 리포트에 별도 표기)
```

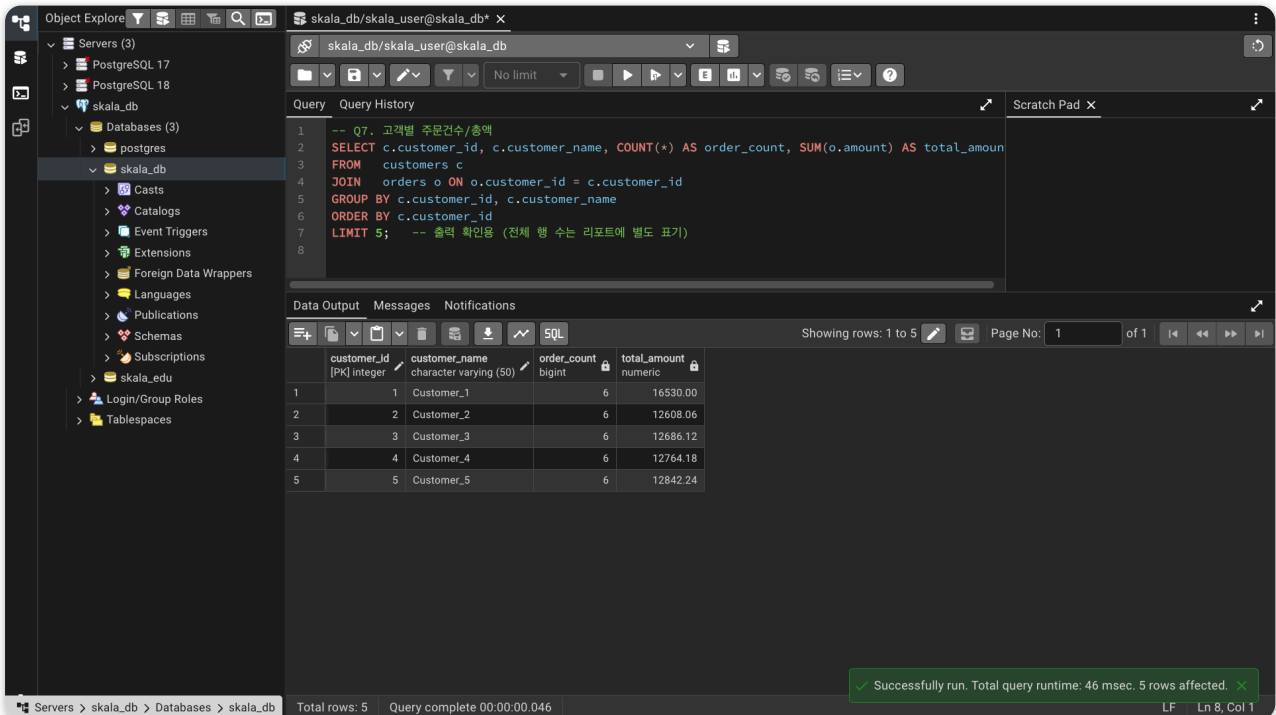
실행 결과 ① — psql

```
scala_db-> \i q07.sql
-- Q7. 고객별 주문건수/총액
SELECT c.customer_id, c.customer_name, COUNT(*) AS order_count, SUM(o.amount) AS total_amount
FROM   customers c
JOIN   orders o ON o.customer_id = c.customer_id
GROUP BY c.customer_id, c.customer_name
ORDER BY c.customer_id
LIMIT 5;  -- 출력 확인용 (전체 행 수는 리포트에 별도 표기 )
 customer_id | customer_name | order_count | total_amount
-----
          1 | Customer_1    |           6 |    16530.00
          2 | Customer_2    |           6 |    12608.06
          3 | Customer_3    |           6 |    12686.12
          4 | Customer_4    |           6 |    12764.18
          5 | Customer_5    |           6 |    12842.24

(5개 행 )

scala_db-> []
```

실행 결과 ② — pgAdmin 4 Query Tool



The screenshot shows the pgAdmin 4 Query Tool interface. The left sidebar displays the database structure, including the 'scala_db' database. The main window shows the query history and the results of the executed query. The query is as follows:

```
-- Q7. 고객별 주문건수/총액
SELECT c.customer_id, c.customer_name, COUNT(*) AS order_count, SUM(o.amount) AS total_amount
FROM   customers c
JOIN   orders o ON o.customer_id = c.customer_id
GROUP BY c.customer_id, c.customer_name
ORDER BY c.customer_id
LIMIT 5;  -- 출력 확인용 (전체 행 수는 리포트에 별도 표기)
```

The results are displayed in a table with 5 rows and 4 columns:

customer_id	customer_name	order_count	total_amount
1	Customer_1	6	16530.00
2	Customer_2	6	12608.06
3	Customer_3	6	12686.12
4	Customer_4	6	12764.18
5	Customer_5	6	12842.24

The status bar at the bottom indicates: 'Total rows: 5 | Query complete 00:00:00.046 | Successfully run. Total query runtime: 46 msec. 5 rows affected.'

고객별 집계 — 교재 배경대로 고객 1명당 정확히 6건의 주문 확인 (전체 500행).

8. 총액 상위 10명과 금액

SQL

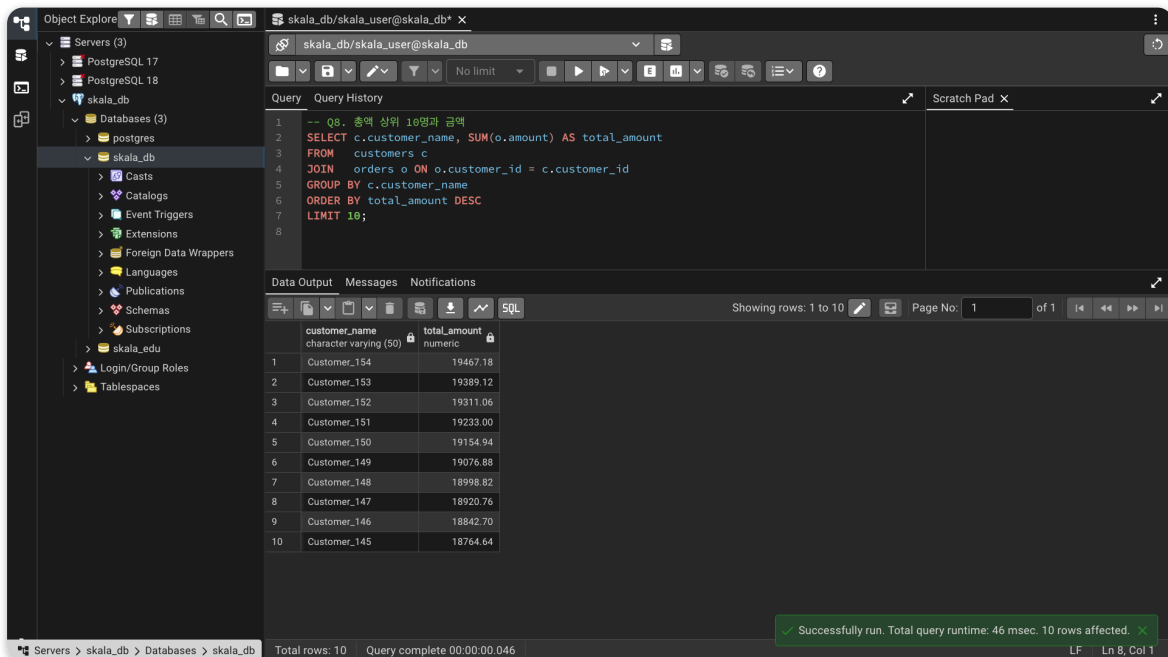
```
-- Q8. 총액 상위 10명과 금액
SELECT c.customer_name, SUM(o.amount) AS total_amount
FROM   customers c
JOIN   orders o ON o.customer_id = c.customer_id
GROUP BY c.customer_name
ORDER BY total_amount DESC
LIMIT 10;
```

실행 결과 ① — psql

```
parkyoungseo — psql -h localhost -U skala_user -d skala_db — 99x29
skala_db-> \i q08.sql
-- Q8. 총액 상위 10명과 금액
SELECT c.customer_name, SUM(o.amount) AS total_amount
FROM   customers c
JOIN   orders o ON o.customer_id = c.customer_id
GROUP BY c.customer_name
ORDER BY total_amount DESC
LIMIT 10;
 customer_name | total_amount
-----
Customer_154   | 19467.18
Customer_153   | 19389.12
Customer_152   | 19311.06
Customer_151   | 19233.00
Customer_150   | 19154.94
Customer_149   | 19076.88
Customer_148   | 18998.82
Customer_147   | 18920.76
Customer_146   | 18842.70
Customer_145   | 18764.64
(10개 행 )

skala_db-> []
```

실행 결과 ② — pgAdmin 4 Query Tool



ORDER BY 총액 DESC + LIMIT 10 — 상위 10명 전체 표시 (문항 지정 건수).

9. 모든 직원과 그 매니저 이름

SQL

```
-- Q9. 모든 직원과 그 매니저 이름 (CEO는 매니저가 없어 NULL, LEFT SELF JOIN)
SELECT e.name AS employee, m.name AS manager
FROM   emp e
LEFT JOIN emp m ON m.emp_id = e.manager_id
ORDER BY e.emp_id
LIMIT 5;  -- 출력 확인용 (전체 311행)
```

실행 결과 ① — psql

```
parkyoungseo — psql -h localhost -U skala_user -d skala_db — 99x22

skala_db-> q
skala_db-> \r
쿼리 버퍼 초기화 (비웠음).
skala_db-> \pset pager off
화면단위 보기 기능 끄 (전체 자료 모두 보여줌).
skala_db-> \! clear

skala_db=> \i q09.sql
-- Q9. 모든 직원과 그 매니저 이름 (CEO는 매니저가 없어 NULL, LEFT SELF JOIN)
SELECT e.name AS employee, m.name AS manager
FROM   emp e
LEFT JOIN emp m ON m.emp_id = e.manager_id
ORDER BY e.emp_id
LIMIT 5;  -- 출력 확인용 (전체 311행)
employee | manager
-----+-----
CEO      |
Mgr_2    | CEO
Mgr_3    | CEO
Mgr_4    | CEO
Mgr_5    | CEO
```

실행 결과 ② — pgAdmin 4 Query Tool

The screenshot shows the pgAdmin 4 Query Tool interface. The query editor contains the same SQL query as in the psql terminal. The query history shows the query was executed successfully. The data output pane displays the results of the query, showing 5 rows of data. The status bar at the bottom indicates the query was successfully run with a total runtime of 48 msec and 5 rows affected.

employee	manager
CEO	[null]
Mgr_2	CEO
Mgr_3	CEO
Mgr_4	CEO
Mgr_5	CEO

LEFT SELF JOIN — 문항 요구대로 직원·매니저 이름만 선택, CEO는 manager가 NULL (전체 311행).

10. “모든 학생 기준”으로 과목 분포 → LEFT JOIN + 집계

SQL

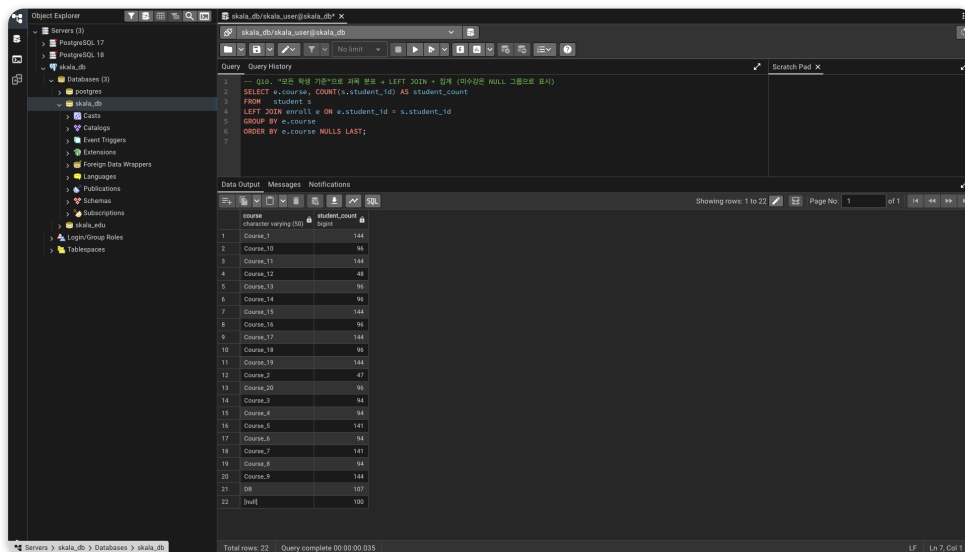
```
-- Q10. "모든 학생 기준"으로 과목 분포 → LEFT JOIN + 집계 (미수강은 NULL 그룹으로 표시)
SELECT e.course, COUNT(s.student_id) AS student_count
FROM   student s
LEFT JOIN enroll e ON e.student_id = s.student_id
GROUP BY e.course
ORDER BY e.course NULLS LAST;
```

실행 결과 ① — psql

```
scala_db-> \i q10.sql
-- Q10. "모든 학생 기준"으로 과목 분포 → LEFT JOIN + 집계 (미수강은 NULL 그룹으로 표시)
SELECT e.course, COUNT(s.student_id) AS student_count
FROM   student s
LEFT JOIN enroll e ON e.student_id = s.student_id
GROUP BY e.course
ORDER BY e.course NULLS LAST;
   course | student_count
-----|-----
 Course_1 |           144
 Course_10 |            96
 Course_11 |           144
 Course_12 |            48
 Course_13 |            96
 Course_14 |            96
 Course_15 |           144
 Course_16 |            96
 Course_17 |           144
 Course_18 |            96
 Course_19 |           144
 Course_2 |            47
 Course_20 |            96
 Course_3 |            94
 Course_4 |            94
 Course_5 |           141
 Course_6 |            94
 Course_7 |           141
 Course_8 |            94
 Course_9 |           144
      DB |           107
      null |           100
(22개 행)

scala_db-> 
```

실행 결과 ② — pgAdmin 4 Query Tool



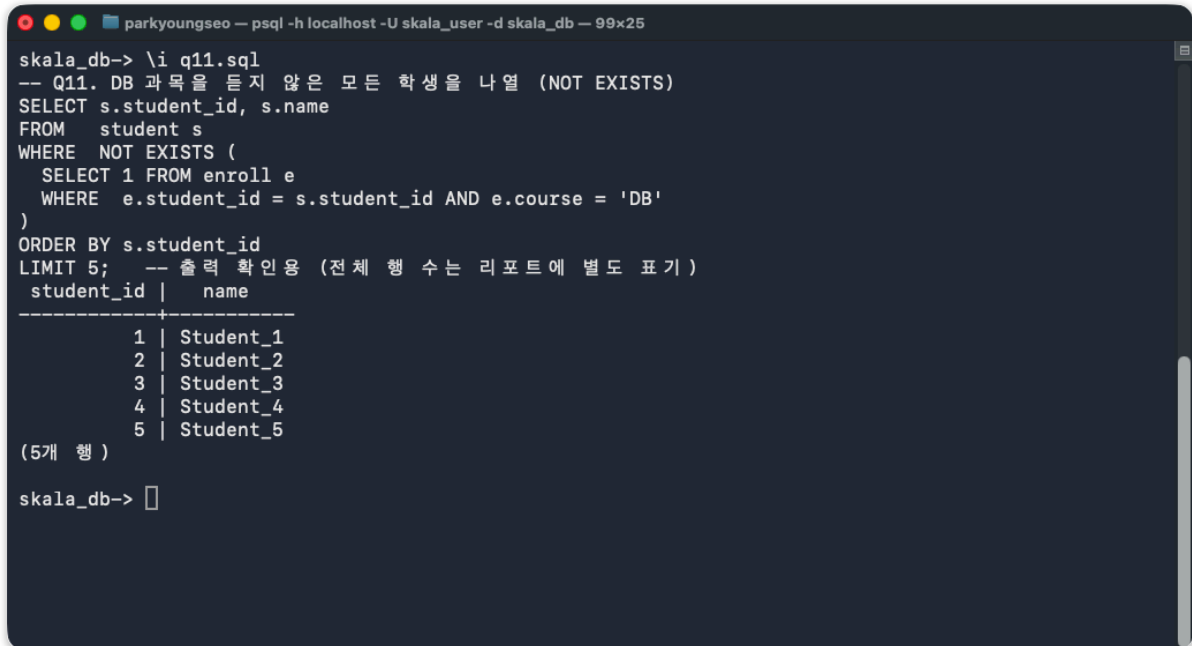
LEFT JOIN 후 과목별 집계 — 미수강 그룹(NULL) 포함 (전체 22행: 수강 과목 21종 + NULL 그룹. 고아 수강 전용 과목 AI·ML은 학생 기준 조인이라 미포함).

11. DB 과목을 듣지 않은 모든 학생을 나열

SQL

```
-- Q11. DB 과목을 듣지 않은 모든 학생을 나열 (NOT EXISTS)
SELECT s.student_id, s.name
FROM   student s
WHERE  NOT EXISTS (
    SELECT 1 FROM enroll e
    WHERE  e.student_id = s.student_id AND e.course = 'DB'
)
ORDER BY s.student_id
LIMIT 5;  -- 출력 확인용 (전체 행 수는 리포트에 별도 표기)
```

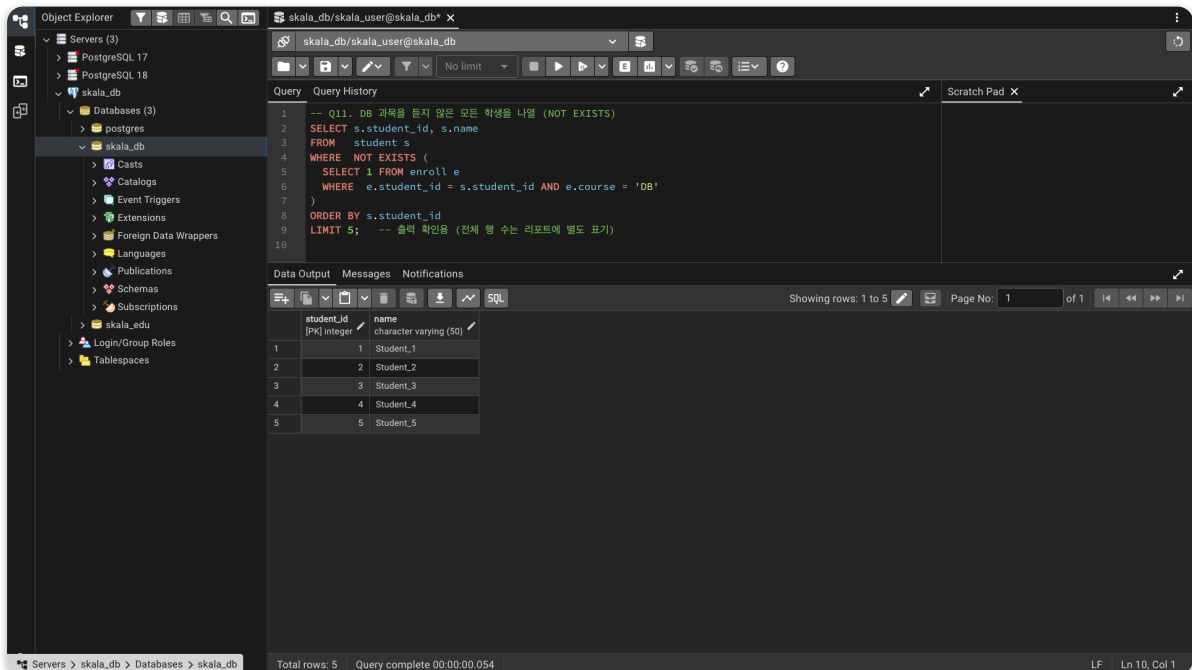
실행 결과 ① — psql



```
parkyoungseo ~ psql -h localhost -U skala_user -d skala_db - 99x25
skala_db-> \i q11.sql
-- Q11. DB 과목을 듣지 않은 모든 학생을 나열 (NOT EXISTS)
SELECT s.student_id, s.name
FROM   student s
WHERE  NOT EXISTS (
    SELECT 1 FROM enroll e
    WHERE  e.student_id = s.student_id AND e.course = 'DB'
)
ORDER BY s.student_id
LIMIT 5;  -- 출력 확인용 (전체 행 수는 리포트에 별도 표기)
 student_id | name
-----+-----
          1 | Student_1
          2 | Student_2
          3 | Student_3
          4 | Student_4
          5 | Student_5
(5개 행)

skala_db-> 
```

실행 결과 ② — pgAdmin 4 Query Tool



The screenshot shows the pgAdmin 4 Query Tool interface. The left sidebar displays the database structure, including the 'skala_db' database. The main pane shows the SQL query being executed, which is the same as the one in the previous block. The 'Data Output' tab at the bottom displays the results of the query, showing 5 rows of student data.

student_id	name
1	Student_1
2	Student_2
3	Student_3
4	Student_4
5	Student_5

NOT EXISTS 안티 조인 — DB 미수강 893명 중 상위 5명.

12. course_owner(course, manager_id) 생성 후 과목별 수강 인원 + 책임 매니저 이름 리포트

SQL

```
-- Q12. 과목별 매니저 운영 책임 가정: course_owner(course, manager_id) 생성 후
--      과목별 수강 인원 + 책임 매니저 이름 리포트
DROP TABLE IF EXISTS course_owner;

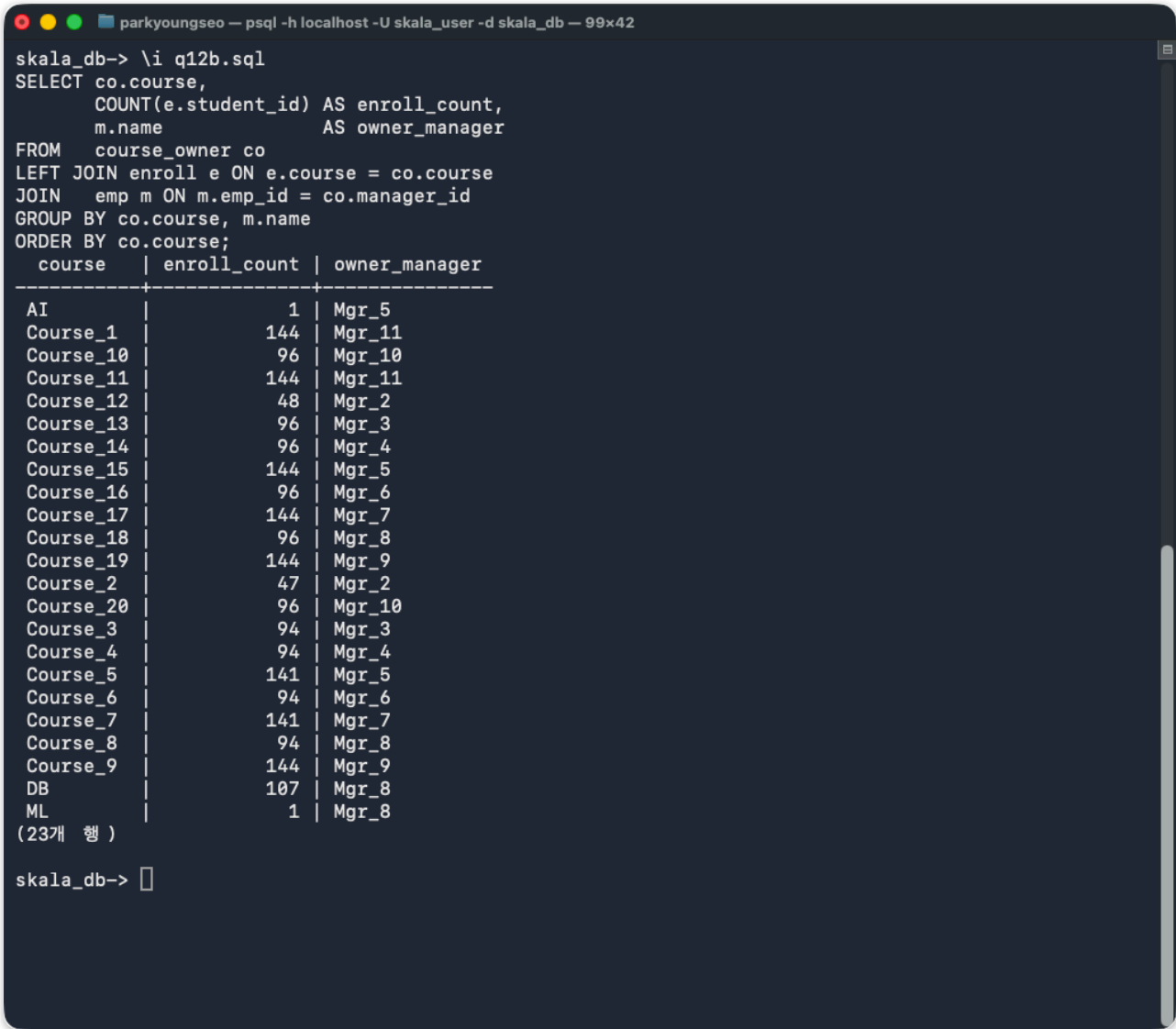
CREATE TABLE course_owner AS
SELECT c.course,
       m.emp_id AS manager_id
FROM   (SELECT DISTINCT course FROM enroll) c
JOIN LATERAL (
  SELECT emp_id
  FROM   emp
  WHERE  name LIKE 'Mgr\_%'
  ORDER BY emp_id
  OFFSET (ascii(right(c.course, 1)) % 10) LIMIT 1  -- 과목→매니저 임의 매핑
) m ON TRUE;

SELECT co.course,
       COUNT(e.student_id) AS enroll_count,
       m.name              AS owner_manager
FROM   course_owner co
LEFT JOIN enroll e ON e.course = co.course
JOIN   emp m ON m.emp_id = co.manager_id
GROUP BY co.course, m.name
ORDER BY co.course;
```

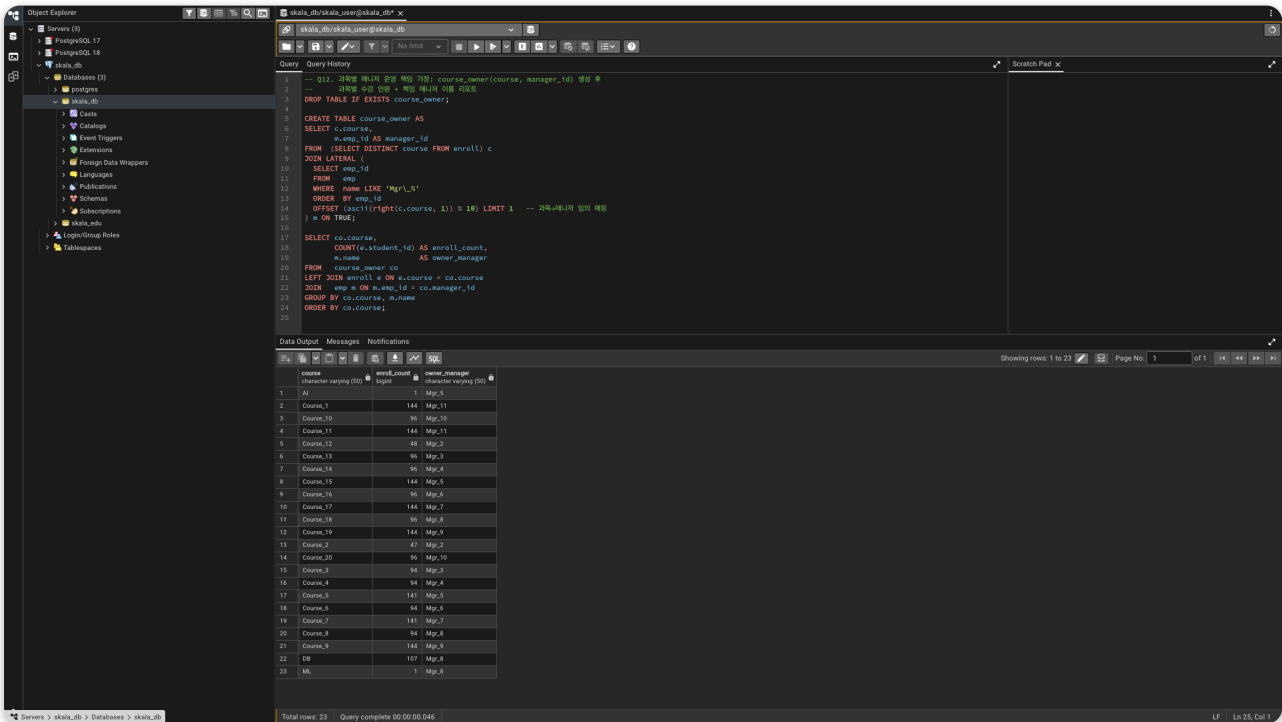
실행 결과 ① — psql (a — course_owner 생성)

```
parkyoungseo — psql -h localhost -U skala_user -d skala_db — 99x28
skala_db-> \i q12a.sql
-- Q12. 과목별 매니저 운영 책임 가정: course_owner(course, manager_id) 생성 후
--      과목별 수강 인원 + 책임 매니저 이름 리포트
DROP TABLE IF EXISTS course_owner;
DROP TABLE
CREATE TABLE course_owner AS
SELECT c.course,
       m.emp_id AS manager_id
FROM   (SELECT DISTINCT course FROM enroll) c
JOIN LATERAL (
  SELECT emp_id
  FROM   emp
  WHERE  name LIKE 'Mgr\_%'
  ORDER BY emp_id
  OFFSET (ascii(right(c.course, 1)) % 10) LIMIT 1  -- 과목→매니저 임의 매핑
) m ON TRUE;
SELECT 23
skala_db-> □
```

실행 결과 ① — psql (b — 과목별 리포트)



실행 결과 ② — pgAdmin 4 Query Tool



(a) CTAS + LATERAL로 course_owner 23건(과목 23종) 생성 → (b) 과목별 수강 인원 + 책임 매니저 리포트 (23행).

13. 학생 × 과목 전체 조합으로 “학생별 과목 추천 후보” 생성, 샘플 100건

SQL

```
-- Q13. 학생 × 과목 전체 조합(CROSS JOIN)으로 "학생별 과목 추천 후보" 생성, 샘플 100건만 조회
SELECT s.student_id, s.name, c.course
FROM   student s
CROSS JOIN (SELECT DISTINCT course FROM enroll) c
ORDER BY s.student_id, c.course
LIMIT 100;
```

실행 결과 ① — psql (쿼리 + 상단)

```
parkyoungseo — psql -h localhost -U skala_user -d skala_db — 110x51

skala_db=> \i q13.sql
-- Q13. 학생 × 과목 전체 조합(CROSS JOIN)으로 "학생별 과목 추천 후보" 생성, 샘플 100건만 조회
SELECT s.student_id, s.name, c.course
FROM   student s
CROSS JOIN (SELECT DISTINCT course FROM enroll) c
ORDER BY s.student_id, c.course
LIMIT 100;
 student_id |   name   | course
-----+-----+-----
          1 | Student_1 | AI
          1 | Student_1 | Course_1
          1 | Student_1 | Course_10
          1 | Student_1 | Course_11
          1 | Student_1 | Course_12
          1 | Student_1 | Course_13
          1 | Student_1 | Course_14
          1 | Student_1 | Course_15
          1 | Student_1 | Course_16
          1 | Student_1 | Course_17
          1 | Student_1 | Course_18
          1 | Student_1 | Course_19
          1 | Student_1 | Course_2
          1 | Student_1 | Course_20
          1 | Student_1 | Course_3
          1 | Student_1 | Course_4
          1 | Student_1 | Course_5
          1 | Student_1 | Course_6
          1 | Student_1 | Course_7
          1 | Student_1 | Course_8
          1 | Student_1 | Course_9
          1 | Student_1 | DB
          1 | Student_1 | ML
          2 | Student_2 | AI
          2 | Student_2 | Course_1
          2 | Student_2 | Course_10
          2 | Student_2 | Course_11
          2 | Student_2 | Course_12
          2 | Student_2 | Course_13
          2 | Student_2 | Course_14
          2 | Student_2 | Course_15
          2 | Student_2 | Course_16
          2 | Student_2 | Course_17
          2 | Student_2 | Course_18
          2 | Student_2 | Course_19
          2 | Student_2 | Course_2
          2 | Student_2 | Course_20
          2 | Student_2 | Course_3
          2 | Student_2 | Course_4
          2 | Student_2 | Course_5
skala_db=> 
```

실행 결과 ① — psql (말미 — 100개 행 확인)

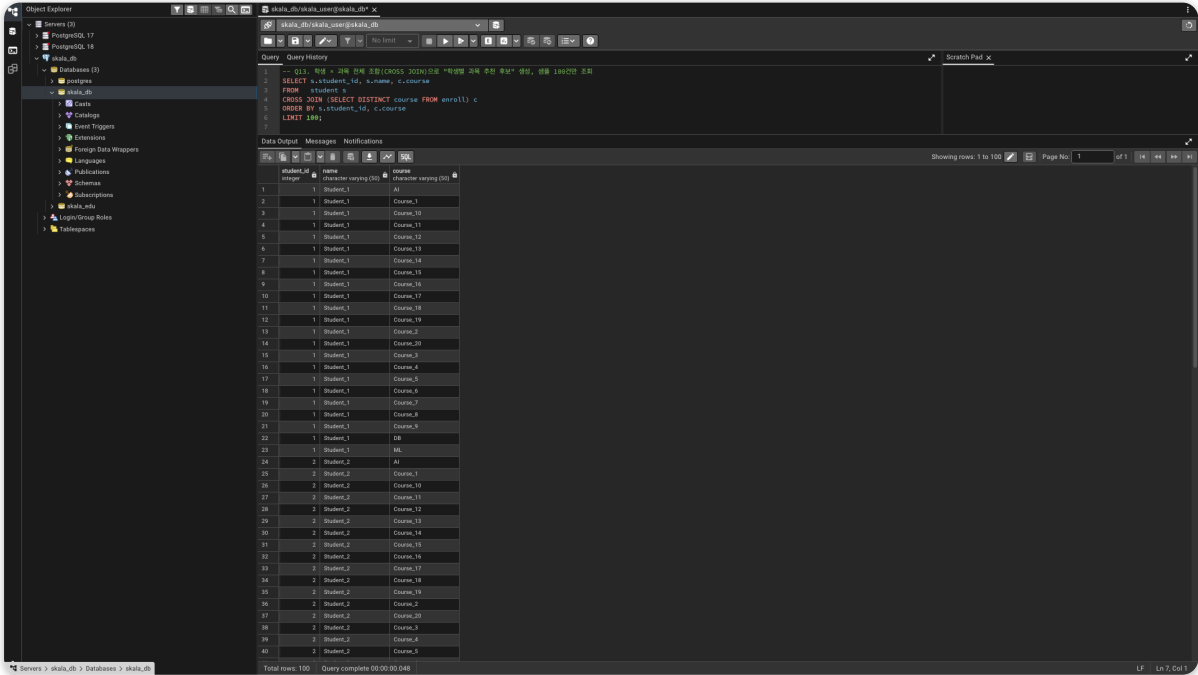
```
parkyoungseo — psql -h localhost -U skala_user -d skala_db — 110x51

3 | Student_3 | Course_14
3 | Student_3 | Course_15
3 | Student_3 | Course_16
3 | Student_3 | Course_17
3 | Student_3 | Course_18
3 | Student_3 | Course_19
3 | Student_3 | Course_2
3 | Student_3 | Course_20
3 | Student_3 | Course_3
3 | Student_3 | Course_4
3 | Student_3 | Course_5
3 | Student_3 | Course_6
3 | Student_3 | Course_7
3 | Student_3 | Course_8
3 | Student_3 | Course_9
3 | Student_3 | DB
3 | Student_3 | ML
4 | Student_4 | AI
4 | Student_4 | Course_1
4 | Student_4 | Course_10
4 | Student_4 | Course_11
4 | Student_4 | Course_12
4 | Student_4 | Course_13
4 | Student_4 | Course_14
4 | Student_4 | Course_15
4 | Student_4 | Course_16
4 | Student_4 | Course_17
4 | Student_4 | Course_18
4 | Student_4 | Course_19
4 | Student_4 | Course_2
4 | Student_4 | Course_20
4 | Student_4 | Course_3
4 | Student_4 | Course_4
4 | Student_4 | Course_5
4 | Student_4 | Course_6
4 | Student_4 | Course_7
4 | Student_4 | Course_8
4 | Student_4 | Course_9
4 | Student_4 | DB
4 | Student_4 | ML
5 | Student_5 | AI
5 | Student_5 | Course_1
5 | Student_5 | Course_10
5 | Student_5 | Course_11
5 | Student_5 | Course_12
5 | Student_5 | Course_13
5 | Student_5 | Course_14
5 | Student_5 | Course_15

(100개 행 )

skala_db=>
```

실행 결과 ② — pgAdmin 4 Query Tool



CROSS JOIN — 학생 1,000명 × 과목 23종(고아 수강의 AI-ML 포함) 조합 중 샘플 100건(문항 지정)만 조회. psql 화면은 쿼리+상단 / 말미 "(100개 행)" 2장 구성.

14. 스칼라 서브쿼리(SELECT 절)로 학생 + 소속 학과명 붙이기

※ 여기서 "붙이기"는 학생 정보와 소속 학과명 컬럼을 연속해서(붙여서) 출력하는 것으로 해석함 — 스칼라 서브쿼리로 구한 학과명(major)을 student_id·name 옆에 컬럼으로 나란히 붙여 출력하였다.

SQL

```
-- Q14. 스칼라 서브쿼리(SELECT 절)로 학생 + 소속 학과명 붙이기
SELECT s.student_id,
       s.name,
       (SELECT s2.major FROM student s2 WHERE s2.student_id = s.student_id) AS major
FROM   student s
ORDER BY s.student_id
LIMIT 5;  -- 출력 확인용 (전체 행 수는 리포트에 별도 표기)
```

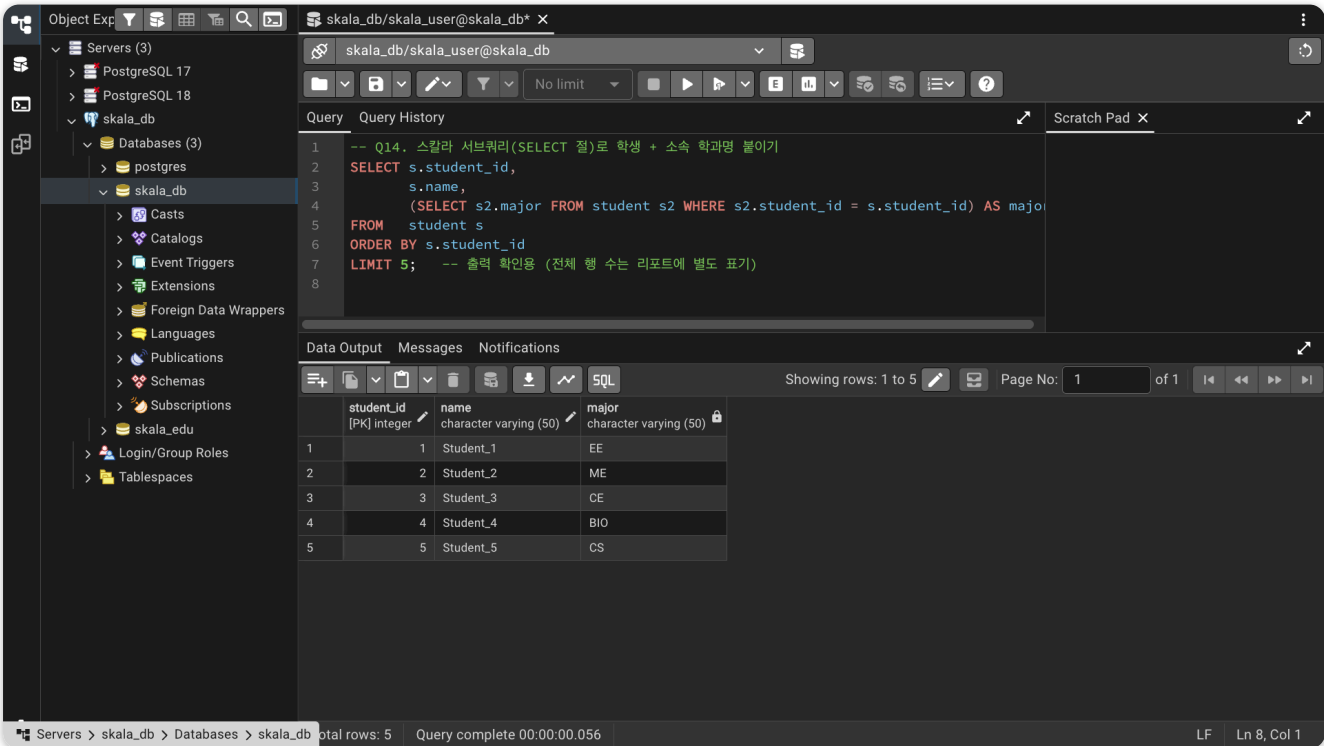
실행 결과 ① — psql

```
parkyoungseo — psql -h localhost -U skala_user -d skala_db -- 99x18

skala_db=> \i q14.sql
-- Q14. 스칼라 서브쿼리(SELECT 절)로 학생 + 소속 학과명 붙이기
SELECT s.student_id,
       s.name,
       (SELECT s2.major FROM student s2 WHERE s2.student_id = s.student_id) AS major
FROM   student s
ORDER BY s.student_id
LIMIT 5;  -- 출력 확인용 (전체 행 수는 리포트에 별도 표기 )
 student_id |  name  | major
-----+-----+-----
          1 | Student_1 | EE
          2 | Student_2 | ME
          3 | Student_3 | CE
          4 | Student_4 | BIO
          5 | Student_5 | CS
(5개 행 )

skala_db=> 
```

실행 결과 ② — pgAdmin 4 Query Tool



The screenshot shows the pgAdmin 4 Query Tool interface. The query editor contains the same SQL query as in the previous block. The results pane shows the output of the query, which is a table with 5 rows and 3 columns: student_id, name, and major.

student_id	name	major
1	Student_1	EE
2	Student_2	ME
3	Student_3	CE
4	Student_4	BIO
5	Student_5	CS

SELECT 절 스칼라 서브쿼리로 행마다 학과명을 붙임 (전체 1,000행).

15. 평균 GPA보다 높은 학생 (WHERE 서브쿼리)

SQL

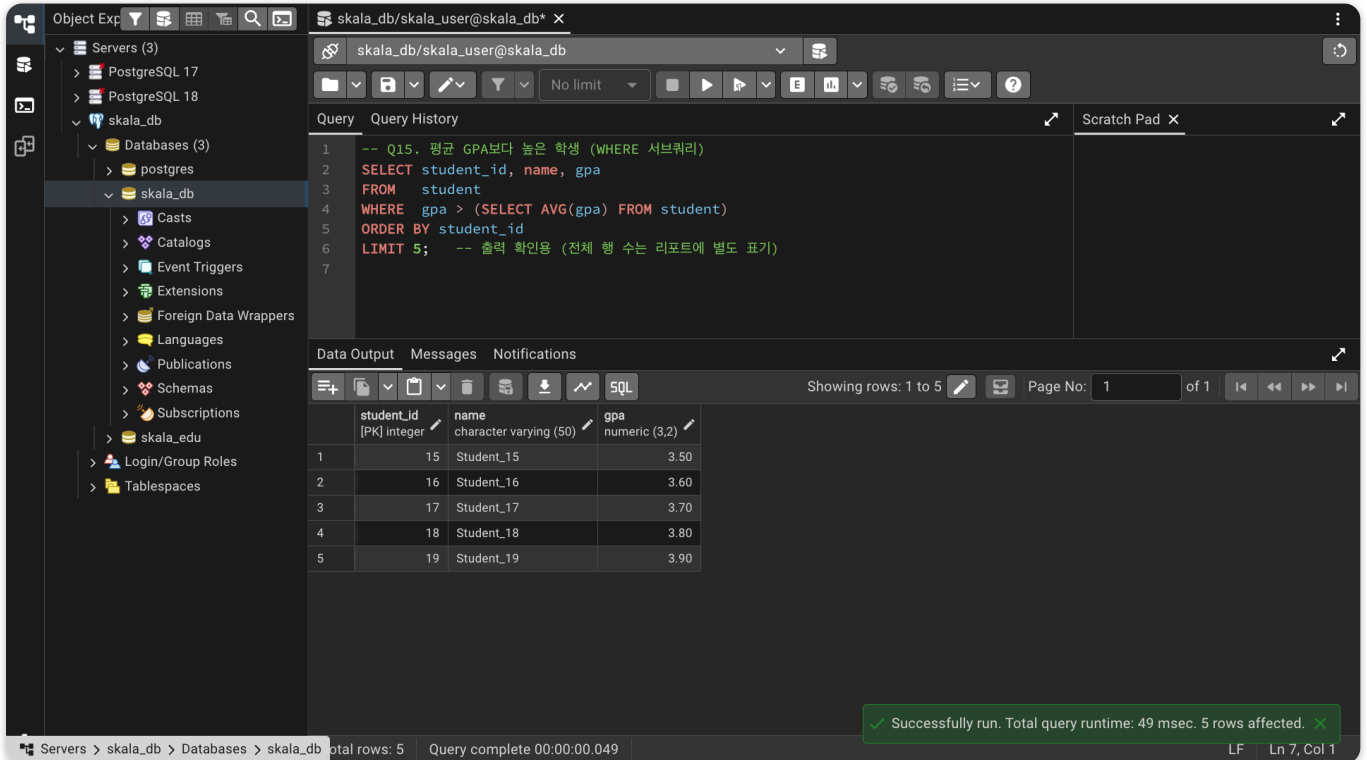
```
-- Q15. 평균 GPA보다 높은 학생 (WHERE 서브쿼리)
SELECT student_id, name, gpa
FROM student
WHERE gpa > (SELECT AVG(gpa) FROM student)
ORDER BY student_id
LIMIT 5;  -- 출력 확인용 (전체 행 수는 리포트에 별도 표기)
```

실행 결과 ① — psql

```
parkyoungseo ~ psql -h localhost -U skala_user -d skala_db - 99x22
skala_db-> \i q15.sql
-- Q15. 평균 GPA보다 높은 학생 (WHERE 서브쿼리)
SELECT student_id, name, gpa
FROM student
WHERE gpa > (SELECT AVG(gpa) FROM student)
ORDER BY student_id
LIMIT 5;  -- 출력 확인용 (전체 행 수는 리포트에 별도 표기)
 student_id |  name   | gpa
-----+-----+-----
          15 | Student_15 | 3.50
          16 | Student_16 | 3.60
          17 | Student_17 | 3.70
          18 | Student_18 | 3.80
          19 | Student_19 | 3.90
(5개 행)

skala_db-> □
```

실행 결과 ② — pgAdmin 4 Query Tool



The screenshot shows the pgAdmin 4 Query Tool interface. The Query History tab displays the following SQL query:

```
-- Q15. 평균 GPA보다 높은 학생 (WHERE 서브쿼리)
SELECT student_id, name, gpa
FROM student
WHERE gpa > (SELECT AVG(gpa) FROM student)
ORDER BY student_id
LIMIT 5;  -- 출력 확인용 (전체 행 수는 리포트에 별도 표기)
```

The Data Output tab shows the results of the query, which are 5 rows of data:

student_id	name	gpa
15	Student_15	3.50
16	Student_16	3.60
17	Student_17	3.70
18	Student_18	3.80
19	Student_19	3.90

A status message at the bottom indicates: "Successfully run. Total query runtime: 49 msec. 5 rows affected."

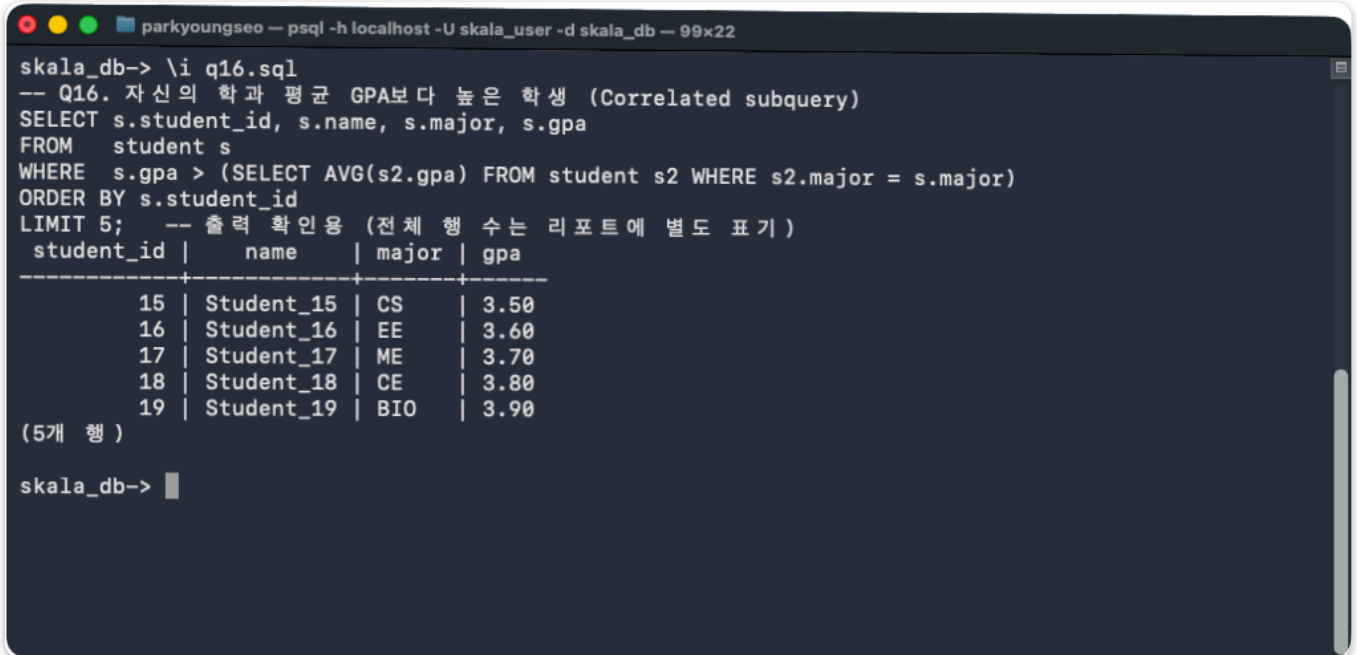
WHERE 서브쿼리 — 전체 평균 GPA(≈3.44) 초과 495명 중 상위 5명.

16. 자신의 학과 평균 GPA보다 높은 학생 (Correlated subquery)

SQL

```
-- Q16. 자신의 학과 평균 GPA보다 높은 학생 (Correlated subquery)
SELECT s.student_id, s.name, s.major, s.gpa
FROM   student s
WHERE  s.gpa > (SELECT AVG(s2.gpa) FROM student s2 WHERE s2.major = s.major)
ORDER BY s.student_id
LIMIT 5;  -- 출력 확인용 (전체 행 수는 리포트에 별도 표기)
```

실행 결과 ① — psql

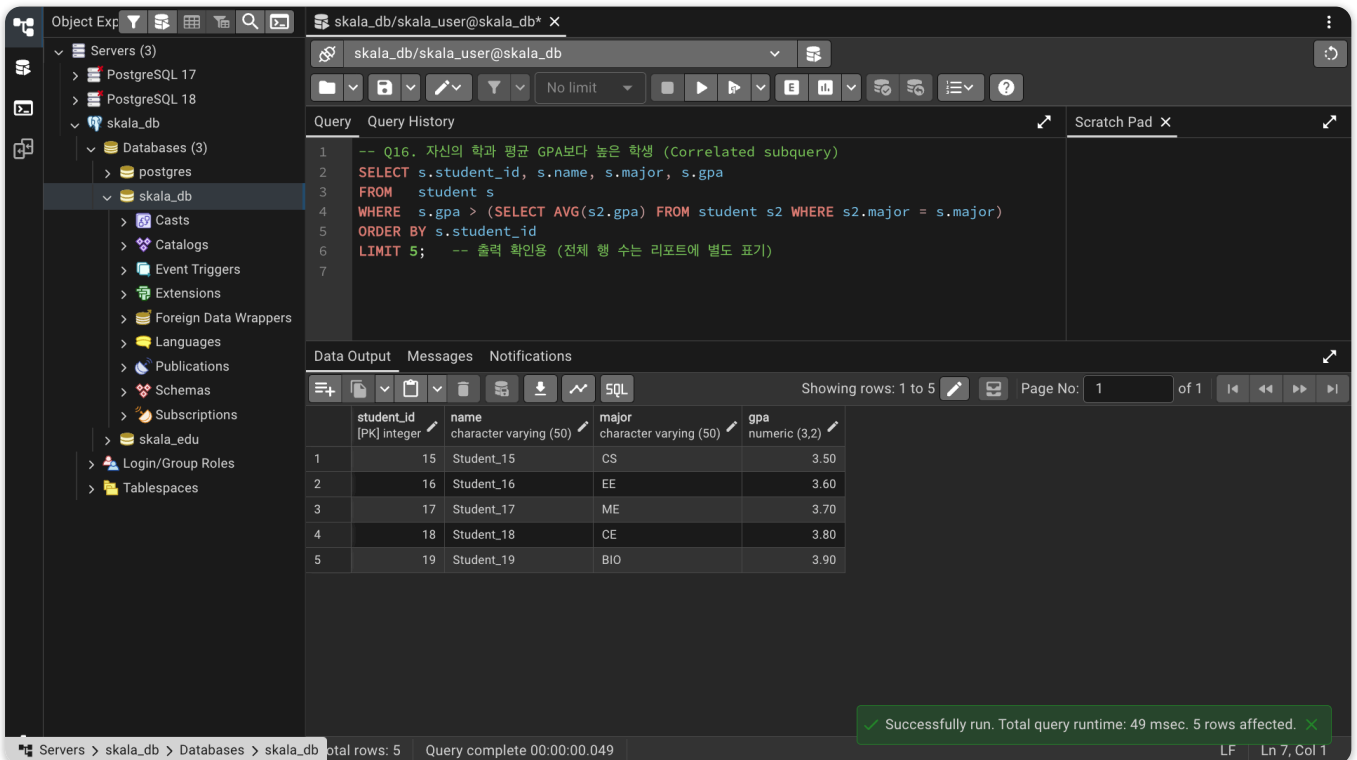


```
skala_db-> \i q16.sql
-- Q16. 자신의 학과 평균 GPA보다 높은 학생 (Correlated subquery)
SELECT s.student_id, s.name, s.major, s.gpa
FROM   student s
WHERE  s.gpa > (SELECT AVG(s2.gpa) FROM student s2 WHERE s2.major = s.major)
ORDER BY s.student_id
LIMIT 5;  -- 출력 확인용 (전체 행 수는 리포트에 별도 표기 )
 student_id |   name   | major | gpa
-----+-----+-----+-----
          15 | Student_15 | CS    | 3.50
          16 | Student_16 | EE    | 3.60
          17 | Student_17 | ME    | 3.70
          18 | Student_18 | CE    | 3.80
          19 | Student_19 | BIO   | 3.90

(5개 행 )

skala_db-> 
```

실행 결과 ② — pgAdmin 4 Query Tool



The screenshot shows the pgAdmin 4 interface with the following components:

- Object Explorer:** Shows the database hierarchy: Servers (3) > PostgreSQL 17 > PostgreSQL 18 > skala_db > Databases (3) > postgres > skala_db > Casts > Catalogs > Event Triggers > Extensions > Foreign Data Wrappers > Languages > Publications > Schemas > skala_edu > Login/Group Roles > Tablespace.
- Query Editor:** Contains the SQL query:

```
-- Q16. 자신의 학과 평균 GPA보다 높은 학생 (Correlated subquery)
SELECT s.student_id, s.name, s.major, s.gpa
FROM   student s
WHERE  s.gpa > (SELECT AVG(s2.gpa) FROM student s2 WHERE s2.major = s.major)
ORDER BY s.student_id
LIMIT 5;  -- 출력 확인용 (전체 행 수는 리포트에 별도 표기)
```
- Data Output:** Displays the results in a table:

student_id	name	major	gpa
1	Student_15	CS	3.50
2	Student_16	EE	3.60
3	Student_17	ME	3.70
4	Student_18	CE	3.80
5	Student_19	BIO	3.90
- Messages:** Shows a green message: "Successfully run. Total query runtime: 49 msec. 5 rows affected."
- Footer:** Shows "Servers > skala_db > Databases > skala_db" and "Query complete 00:00:00.049".

상관 서브쿼리 — 행마다 자기 학과 평균과 비교, 495명 중 상위 5명.

17. 수강(enroll) 기록이 있는 학생만 (EXISTS)

SQL

```
-- Q17. 수강(enroll) 기록이 있는 학생만 (EXISTS)
SELECT s.student_id, s.name
FROM   student s
WHERE  EXISTS (SELECT 1 FROM enroll e WHERE e.student_id = s.student_id)
ORDER BY s.student_id
LIMIT 5;  -- 출력 확인용 (전체 행 수는 리포트에 별도 표기)
```

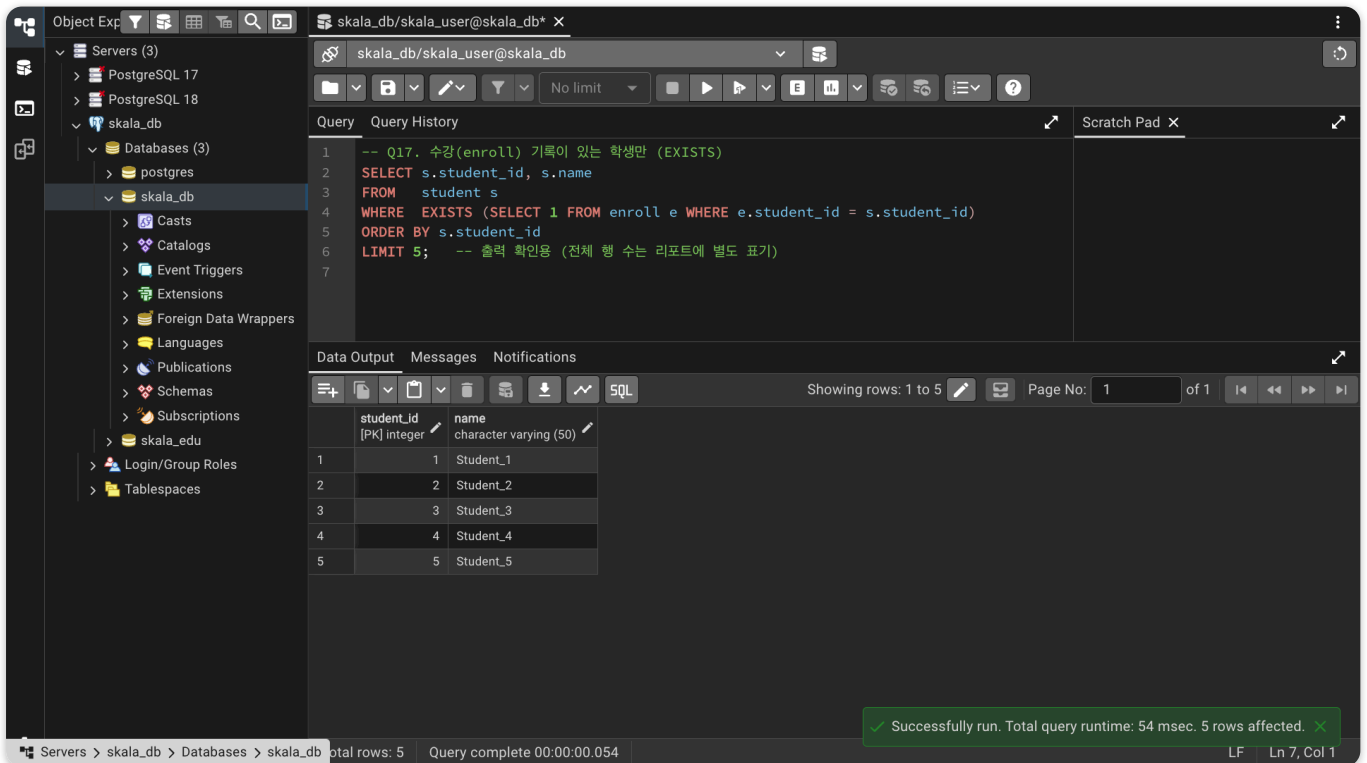
실행 결과 ① — psql

```
parkyoungseo — psql -h localhost -U skala_user -d skala_db — 99x22

skala_db-> \i q17.sql
-- Q17. 수강(enroll) 기록이 있는 학생만 (EXISTS)
SELECT s.student_id, s.name
FROM   student s
WHERE  EXISTS (SELECT 1 FROM enroll e WHERE e.student_id = s.student_id)
ORDER BY s.student_id
LIMIT 5;  -- 출력 확인용 (전체 행 수는 리포트에 별도 표기)
 student_id | name
-----+-----
          1 | Student_1
          2 | Student_2
          3 | Student_3
          4 | Student_4
          5 | Student_5
(5개 행)

skala_db->
```

실행 결과 ② — pgAdmin 4 Query Tool



The screenshot shows the pgAdmin 4 Query Tool interface. The left sidebar displays the database structure, including the 'skala_db' database. The main window shows the 'Query History' tab with the SQL query from the previous block. Below the query, the 'Data Output' tab displays the results of the query. The results are shown in a table with columns 'student_id' and 'name'. The table contains 5 rows of data, corresponding to the students enrolled in the course.

student_id	name
1	Student_1
2	Student_2
3	Student_3
4	Student_4
5	Student_5

At the bottom of the interface, a status bar indicates: 'Successfully run. Total query runtime: 54 msec. 5 rows affected.'

EXISTS 세미 조인 — Q6과 동일 결과(900명)를 다른 기법으로 구현.

18. 한 번도 수강하지 않은 학생 (NOT EXISTS)

SQL

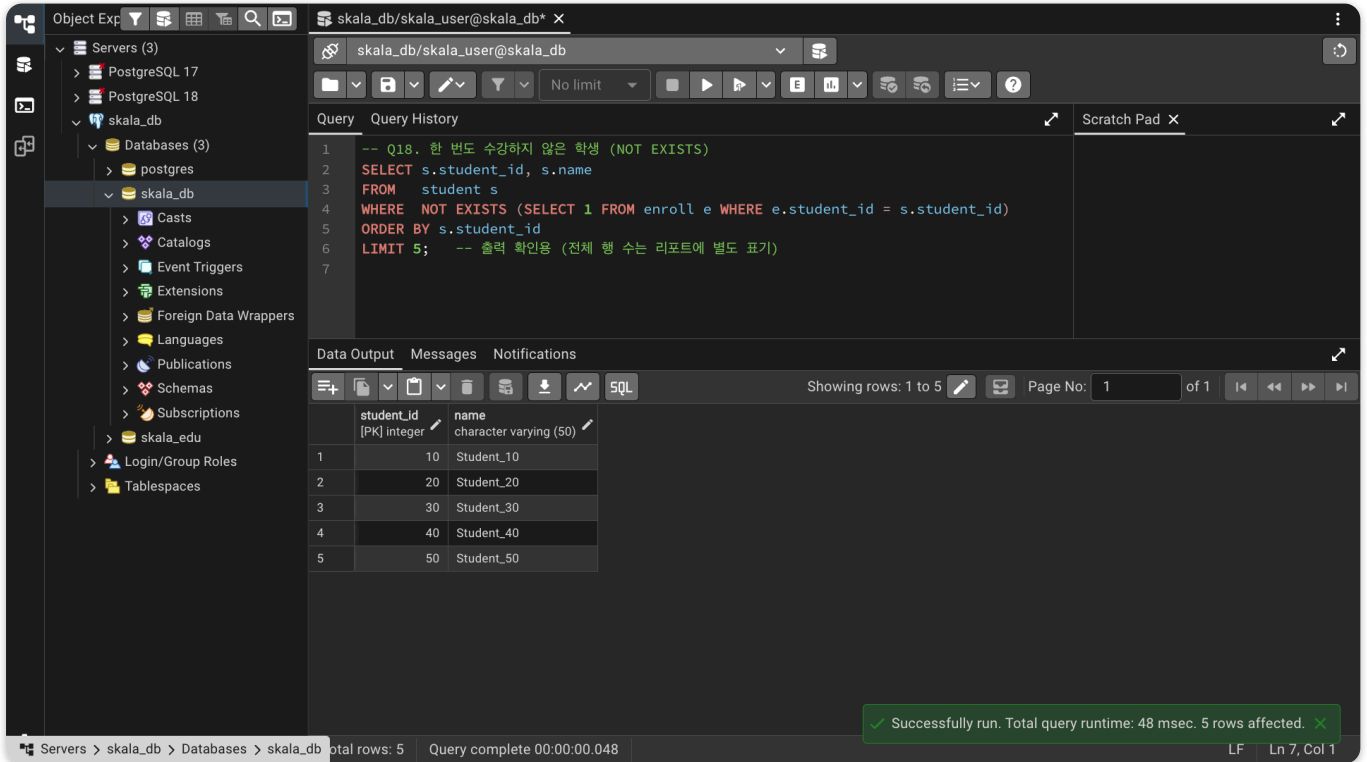
```
-- Q18. 한 번도 수강하지 않은 학생 (NOT EXISTS)
SELECT s.student_id, s.name
FROM   student s
WHERE  NOT EXISTS (SELECT 1 FROM enroll e WHERE e.student_id = s.student_id)
ORDER BY s.student_id
LIMIT 5;  -- 출력 확인용 (전체 행 수는 리포트에 별도 표기)
```

실행 결과 ① — psql

```
parkyoungseo ~ psql -h localhost -U skala_user -d skala_db - 99x22
skala_db-> \i q18.sql
-- Q18. 한 번도 수강하지 않은 학생 (NOT EXISTS)
SELECT s.student_id, s.name
FROM   student s
WHERE  NOT EXISTS (SELECT 1 FROM enroll e WHERE e.student_id = s.student_id)
ORDER BY s.student_id
LIMIT 5;  -- 출력 확인용 (전체 행 수는 리포트에 별도 표기)
 student_id |  name
-----+-----
          10 | Student_10
          20 | Student_20
          30 | Student_30
          40 | Student_40
          50 | Student_50
(5개 행)

skala_db-> 
```

실행 결과 ② — pgAdmin 4 Query Tool



The screenshot shows the pgAdmin 4 Query Tool interface. The Query History tab displays the SQL query: `-- Q18. 한 번도 수강하지 않은 학생 (NOT EXISTS)`, `SELECT s.student_id, s.name`, `FROM student s`, `WHERE NOT EXISTS (SELECT 1 FROM enroll e WHERE e.student_id = s.student_id)`, `ORDER BY s.student_id`, `LIMIT 5;`. The Data Output tab shows the results of the query, which are 5 rows of data. The results table has two columns: `student_id` (integer) and `name` (character varying (50)). The data is as follows:

student_id	name
10	Student_10
20	Student_20
30	Student_30
40	Student_40
50	Student_50

At the bottom of the interface, a green status bar indicates: `Successfully run. Total query runtime: 48 msec. 5 rows affected.`

NOT EXISTS — Q5와 동일 결과(100명)를 다른 기법으로 구현.

19. HR 학과 학생 일부와의 비교 데모 (> ANY)

SQL

```
-- Q19. HR 학과 학생 일부와의 비교 데모 (> ANY: HR 학생 중 한 명보다라도 GPA가 높으면 포함)
SELECT student_id, name, gpa
FROM student
WHERE gpa > ANY (SELECT gpa FROM student WHERE major = 'HR')
ORDER BY student_id
LIMIT 5;  -- 출력 확인용 (전체 행 수는 리포트에 별도 표기)
```

실행 결과 ① — psql

```
parkyoungseo — psql -h localhost -U skala_user -d skala_db — 99x22
skala_db-> \i q19.sql
-- Q19. HR 학과 학생 일부와의 비교 데모 (> ANY: HR 학생 중 한 명보다라도 GPA가 높으면 포함)
SELECT student_id, name, gpa
FROM student
WHERE gpa > ANY (SELECT gpa FROM student WHERE major = 'HR')
ORDER BY student_id
LIMIT 5;  -- 출력 확인용 (전체 행 수는 리포트에 별도 표기)
 student_id | name      | gpa
-----+-----+-----
          1 | Student_1 | 2.10
          2 | Student_2 | 2.20
          3 | Student_3 | 2.30
          4 | Student_4 | 2.40
          5 | Student_5 | 2.50
(5개 행)
skala_db-> □
```

실행 결과 ② — pgAdmin 4 Query Tool

The screenshot shows the pgAdmin 4 Query Tool interface. The left sidebar displays the database structure, including the 'skala_db' database. The main pane shows the SQL query being executed. The 'Data Output' tab is active, displaying the results of the query in a table format. The table has three columns: 'student_id', 'name', and 'gpa'. The results show the top 5 students by GPA who are higher than any HR student's GPA.

student_id	name	gpa
1	Student_1	2.10
2	Student_2	2.20
3	Student_3	2.30
4	Student_4	2.40
5	Student_5	2.50

Successfully run. Total query runtime: 62 msec. 5 rows affected.

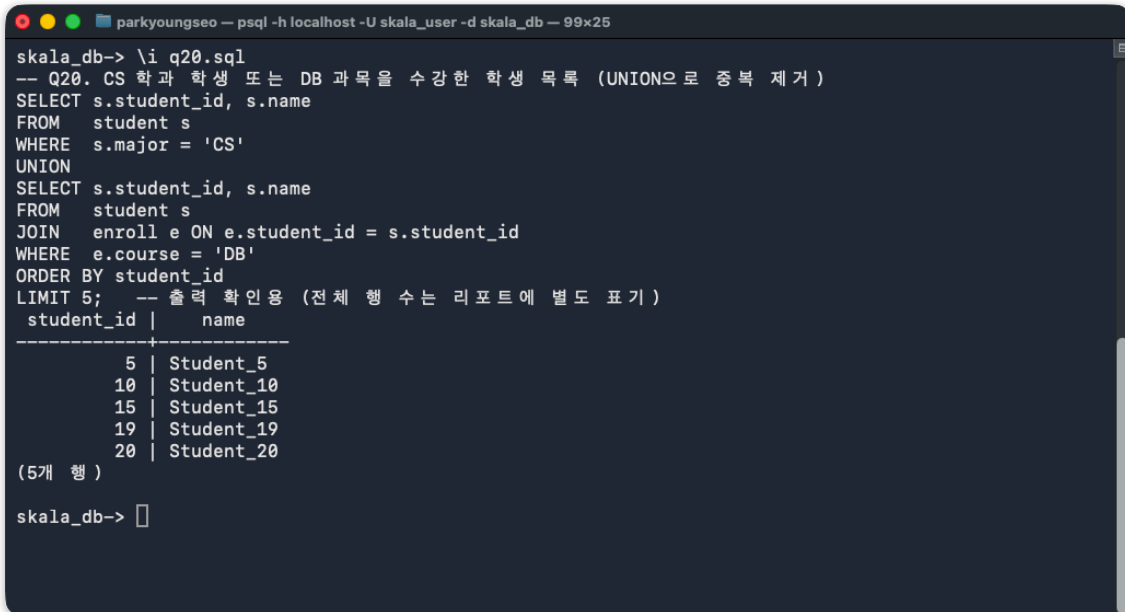
> ANY — HR 학과 최저 GPA(2.0)보다 높으면 포함, 967명 중 상위 5명.

20. CS 학과 학생 또는 DB 과목을 수강한 학생 목록 (UNION)

SQL

```
-- Q20. CS 학과 학생 또는 DB 과목을 수강한 학생 목록 (UNION으로 중복 제거)
SELECT s.student_id, s.name
FROM   student s
WHERE  s.major = 'CS'
UNION
SELECT s.student_id, s.name
FROM   student s
JOIN   enroll e ON e.student_id = s.student_id
WHERE  e.course = 'DB'
ORDER BY student_id
LIMIT 5;  -- 출력 확인용 (전체 행 수는 리포트에 별도 표기)
```

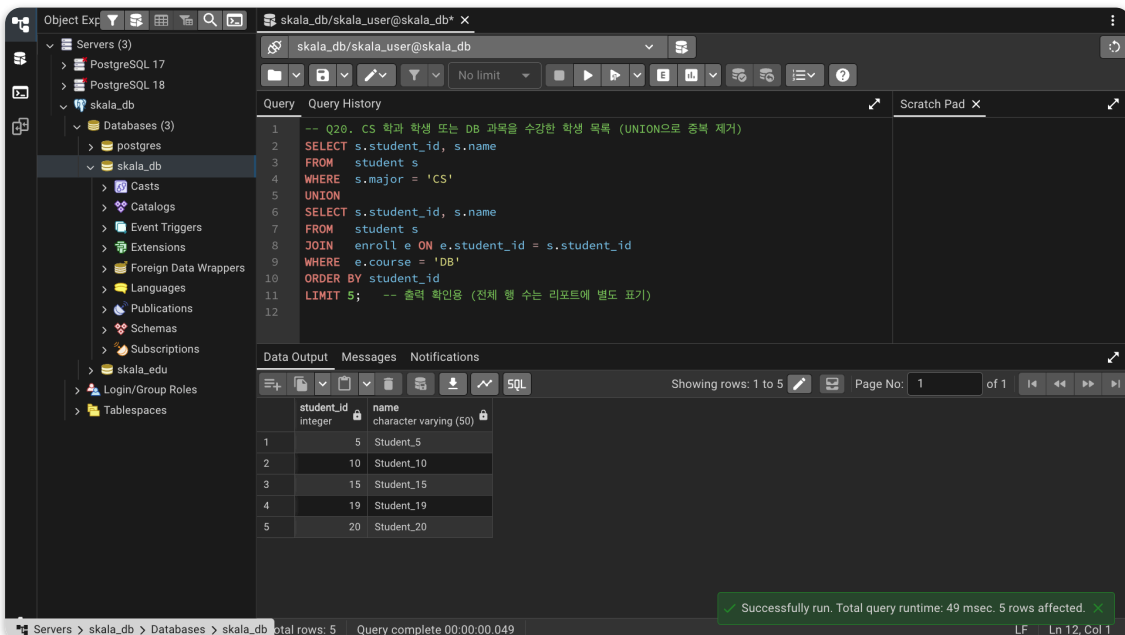
실행 결과 ① — psql



```
scala_db-> \i q20.sql
-- Q20. CS 학과 학생 또는 DB 과목을 수강한 학생 목록 (UNION으로 중복 제거)
SELECT s.student_id, s.name
FROM   student s
WHERE  s.major = 'CS'
UNION
SELECT s.student_id, s.name
FROM   student s
JOIN   enroll e ON e.student_id = s.student_id
WHERE  e.course = 'DB'
ORDER BY student_id
LIMIT 5;  -- 출력 확인용 (전체 행 수는 리포트에 별도 표기)
 student_id | name
-----
          5 | Student_5
         10 | Student_10
         15 | Student_15
         19 | Student_19
         20 | Student_20
(5개 행)

scala_db-> 
```

실행 결과 ② — pgAdmin 4 Query Tool



The screenshot shows the pgAdmin 4 Query Tool interface. The query editor contains the same SQL query as in the psql terminal. The Data Output tab shows the results of the query, which are 5 rows of student data.

student_id	name
5	Student_5
10	Student_10
15	Student_15
19	Student_19
20	Student_20

Successfully run. Total query runtime: 49 msec. 5 rows affected.

UNION — CS 학과(196명) U DB 수강생 = 중복 제거 후 290명 중 상위 5명.

21. 학과별·GPA 구간별 인원 집계 — ROLLUP 소계/총계 + GROUPING 라벨

SQL

```
-- Q21. 학과별·GPA 구간별 인원 집계 + ROLLUP 소계/총계
--   - gpa_tier 파생 컬럼(3.0 미만 / 3.0~3.5 / 3.5 초과)
--   - GROUP BY ROLLUP(major, gpa_tier), GROUPING(major)으로 '전체' 라벨
--   - 소계 행은 하단 정렬

SELECT CASE WHEN GROUPING(t.major) = 1 THEN '전체' ELSE t.major END AS major,
       CASE WHEN GROUPING(t.gpa_tier) = 1 THEN '소계' ELSE t.gpa_tier END AS gpa_tier,
       COUNT(*) AS student_count
FROM (
  SELECT major,
         CASE WHEN gpa < 3.0 THEN '3.0 미만'
              WHEN gpa <= 3.5 THEN '3.0~3.5'
              ELSE '3.5 초과'
         END AS gpa_tier
  FROM student
) t
GROUP BY ROLLUP(t.major, t.gpa_tier)
ORDER BY GROUPING(t.major), t.major, GROUPING(t.gpa_tier), t.gpa_tier;
```

실행 결과 ① — psql

```
parkyoungseo — psql -h localhost -U skala_user -d skala_db — 124x56

skala_db-> \i q21.sql
-- Q21. 학과별·GPA 구간별 인원 집계 + ROLLUP 소계/총계
--   - gpa_tier 파생 컬럼(3.0 미만 / 3.0~3.5 / 3.5 초과)
--   - GROUP BY ROLLUP(major, gpa_tier), GROUPING(major)으로 '전체' 라벨
--   - 소계 행은 하단 정렬
SELECT CASE WHEN GROUPING(t.major) = 1 THEN '전체' ELSE t.major END AS major,
       CASE WHEN GROUPING(t.gpa_tier) = 1 THEN '소계' ELSE t.gpa_tier END AS gpa_tier,
       COUNT(*) AS student_count
FROM (
  SELECT major,
         CASE WHEN gpa < 3.0 THEN '3.0 미만'
              WHEN gpa <= 3.5 THEN '3.0~3.5'
              ELSE '3.5 초과'
         END AS gpa_tier
  FROM student
) t
GROUP BY ROLLUP(t.major, t.gpa_tier)
ORDER BY GROUPING(t.major), t.major, GROUPING(t.gpa_tier), t.gpa_tier;
major | gpa_tier | student_count
-----+-----+-----
BIO   | 3.0 미만 |          66
BIO   | 3.0~3.5  |          33
BIO   | 3.5 초과 |          97
BIO   | 소계     |         196
CE     | 3.0 미만 |          66
CE     | 3.0~3.5  |          33
CE     | 3.5 초과 |          97
CE     | 소계     |         196
CS     | 3.0 미만 |          65
CS     | 3.0~3.5  |          66
CS     | 3.5 초과 |          65
CS     | 소계     |         196
EE     | 3.0 미만 |          66
EE     | 3.0~3.5  |          33
EE     | 3.5 초과 |          97
EE     | 소계     |         196
HR     | 3.0 미만 |          10
HR     | 3.0~3.5  |           1
HR     | 3.5 초과 |           9
HR     | 소계     |          20
ME     | 3.0 미만 |          66
ME     | 3.0~3.5  |          33
ME     | 3.5 초과 |          97
ME     | 소계     |         196
전체  | 소계     |        1000
(25개 행)

skala_db-> □
```

실행 결과 ② — pgAdmin 4 Query Tool

Object Explorer

Servers (3)

PostgreSQL 17

PostgreSQL 18

skala_db

Databases (3)

postgres

skala_db

Casts

Catalogs

Event Triggers

Extensions

Foreign Data Wrappers

Languages

Publications

Schemas

Subscriptions

Table Functions

Login/Group Roles

Tablespaces

skala_db/skala_user/skala_db

Query

QueryHistory

Scratch Pad

```

1  -- Q21. 학생별 GPA 구간별 인원 집계 + ROLLUP 소개/총계
2  --
3  --   gpa_tier  학생 등급(3.0 미만 / 3.0-3.5 / 3.5 초과)
4  --   --
5  SELECT CASE WHEN GROUPING(t.major) = 1 THEN '전체' ELSE t.major   END AS major,
6         CASE WHEN GROUPING(t.gpa_tier) = 1 THEN '소개' ELSE t.gpa_tier END AS gpa_tier,
7         COUNT(*) AS student_count
8  FROM (
9         SELECT major,
10                CASE WHEN gpa < 3.0 THEN '3.0 미만'
11                     WHEN gpa <= 3.5 THEN '3.0-3.5'
12                     ELSE '3.5 초과'
13                END AS gpa_tier
14        FROM student
15       ) t
16 GROUP BY ROLLUP(t.major, t.gpa_tier)
17 ORDER BY GROUPING(t.major), t.major, GROUPING(t.gpa_tier), t.gpa_tier;
18

```

Data Output

Messages

Notifications

Showing rows: 1 to 25

Page No: 1

of 1

14

44

44

	major	character varying	gpa_tier	text	student_count	bigint
1	INFO		3.0 미만		66	
2	INFO		3.0-3.5		33	
3	INFO		3.5 초과		97	
4	INFO		소개		196	
5	CE		3.0 미만		66	
6	CE		3.0-3.5		33	
7	CE		3.5 초과		97	
8	CE		소개		196	
9	CS		3.0 미만		65	
10	CS		3.0-3.5		66	
11	CS		3.5 초과		65	
12	CS		소개		196	
13	EE		3.0 미만		66	
14	EE		3.0-3.5		33	
15	EE		3.5 초과		97	
16	EE		소개		196	
17	HR		3.0 미만		10	
18	HR		3.0-3.5		1	
19	HR		3.5 초과		9	
20	HR		소개		20	
21	ME		3.0 미만		66	
22	ME		3.0-3.5		33	
23	ME		3.5 초과		97	
24	ME		소개		196	
25	전체		소개		1000	

Total rows: 25

Query complete 00:00:00.051

Servers > skala_db > Databases > skala_db

LF Ln 18, Col 1

ROLLUP(major, gpa_tier) + GROUPING — 학과×구간 + 학과 소계 + 총계('전체') 25행, 소계는 하단 정렬.

22. WITH RECURSIVE 조직 트리 (depth/path) + 매니저별 direct_reports

SQL

```
-- Q22. WITH RECURSIVE 조직 트리: 모든 직원의 계층 경로(path)와 깊이(depth)
WITH RECURSIVE org AS (
  SELECT emp_id, name, manager_id, 0 AS depth, name::text AS path
  FROM   emp
  WHERE  manager_id IS NULL          -- CEO에서 시작
  UNION ALL
  SELECT e.emp_id, e.name, e.manager_id, o.depth + 1, o.path || ' > ' || e.name
  FROM   emp e
  JOIN   org o ON o.emp_id = e.manager_id
)
SELECT emp_id, name, depth, path
FROM   org
ORDER BY path
LIMIT 5;  -- 출력 확인용 (전체 311행)

-- Q22-2. 매니저별 직속 부하 직원 수 (direct_reports)
SELECT m.name AS manager, COUNT(*) AS direct_reports
FROM   emp e
JOIN   emp m ON m.emp_id = e.manager_id
GROUP BY m.name
ORDER BY direct_reports DESC, m.name;
```

실행 결과 ① — psql (쿼리 1 — 조직 트리)

```
parkyoungseo — psql -h localhost -U skala_user -d skala_db — 99x30

skala_db-> \i q22a.sql
-- Q22. WITH RECURSIVE 조직 트리: 모든 직원의 계층 경로 (path)와 깊이 (depth)
WITH RECURSIVE org AS (
  SELECT emp_id, name, manager_id, 0 AS depth, name::text AS path
  FROM   emp
  WHERE  manager_id IS NULL          -- CEO에서 시작
  UNION ALL
  SELECT e.emp_id, e.name, e.manager_id, o.depth + 1, o.path || ' > ' || e.name
  FROM   emp e
  JOIN   org o ON o.emp_id = e.manager_id
)
SELECT emp_id, name, depth, path
FROM   org
ORDER BY path
LIMIT 5;  -- 출력 확인용 (전체 311행)
 emp_id | name      | depth | path
-----+-----+-----+-----
      1 | CEO      |      0 | CEO
     102 | Dev_102  |      1 | CEO > Dev_102
     112 | Dev_112  |      1 | CEO > Dev_112
      12 | Dev_12   |      1 | CEO > Dev_12
     122 | Dev_122  |      1 | CEO > Dev_122
(5개 행)

skala_db-> 
```

실행 결과 ① — psql (쿼리 2 — direct_reports)

```
scala_db-> \i q22b.sql
-- Q22-2. 매니저별 직속 부하 직원 수 (direct_reports)
SELECT m.name AS manager, COUNT(*) AS direct_reports
FROM   emp e
JOIN   emp m ON m.emp_id = e.manager_id
GROUP BY m.name
ORDER BY direct_reports DESC, m.name;
manager | direct_reports
-----+-----
CEO      |              40
Mgr_10   |              30
Mgr_2    |              30
Mgr_3    |              30
Mgr_4    |              30
Mgr_5    |              30
Mgr_6    |              30
Mgr_7    |              30
Mgr_8    |              30
Mgr_9    |              30
(10개 행 )

scala_db-> []
```

실행 결과 ② — pgAdmin 4 Query Tool (위: 쿼리 1 · 아래: 쿼리 2)

Object Explorer

scala_db/skala_user@skala_db

Query

```
-- Q22. WITH RECURSIVE 조직 트리: 모든 직원의 계층 경로(path)와 깊이(depth)
WITH RECURSIVE org AS (
  SELECT emp_id, name, manager_id, 0 AS depth, name::text AS path
  FROM   emp
  WHERE  manager_id IS NULL -- CEO에서 시작
  UNION ALL
  SELECT e.emp_id, e.name, e.manager_id, o.depth + 1, o.path || ' > ' || e.name
  FROM   emp e
  JOIN   org o ON o.emp_id = e.manager_id
)
SELECT emp_id, name, depth, path
FROM   org
ORDER BY path
LIMIT 5; -- 출력 확인용 (전체 311행)
```

Data Output

emp_id	name	depth	path
1	CEO	0	CEO
102	Dev_102	1	CEO > Dev_1...
112	Dev_112	1	CEO > Dev_1...
12	Dev_12	1	CEO > Dev_12
122	Dev_122	1	CEO > Dev_1...

Total rows: 5 Query complete 00:00:00.044

Object Explorer

scala_db/skala_user@skala_db

Query

```
-- Q22-2. 매니저별 직속 부하 직원 수 (direct_reports)
SELECT m.name AS manager, COUNT(*) AS direct_reports
FROM   emp e
JOIN   emp m ON m.emp_id = e.manager_id
GROUP BY m.name
ORDER BY direct_reports DESC, m.name;
```

Data Output

manager	direct_reports
CEO	40
Mgr_10	30
Mgr_2	30
Mgr_3	30
Mgr_4	30
Mgr_5	30
Mgr_6	30
Mgr_7	30
Mgr_8	30
Mgr_9	30

Total rows: 10 Query complete 00:00:00.049

Successfully run. Total query runtime: 49 msec. 10 rows affected.

(a) WITH RECURSIVE — depth 0=CEO, path 'CEO > ...' 형식 (전체 311행) → (b) 매니저별 direct_reports 집계 (10행).

23. 학과별 GPA 상위 3명 — Window Function (서브쿼리 방식 + CTE 방식)

SQL — 방식 1 (서브쿼리)

```
-- Q23. 학과별 GPA 상위 3명 - Window Function (서브쿼리 방식과 CTE 방식 모두 작성)
--      ROW_NUMBER(동점 시 student_id 2차) + RANK/DENSE_RANK 비교 + COUNT() OVER

-- [방식 1] 서브쿼리 방식
SELECT student_id, name, major, gpa, rn, rnk, drnk, total_in_major
FROM (
    SELECT student_id, name, major, gpa,
           ROW_NUMBER() OVER (PARTITION BY major ORDER BY gpa DESC, student_id) AS rn,
           RANK()       OVER (PARTITION BY major ORDER BY gpa DESC)           AS rnk,
           DENSE_RANK() OVER (PARTITION BY major ORDER BY gpa DESC)           AS drnk,
           COUNT(*)      OVER (PARTITION BY major)                           AS total_in_major
    FROM student
) x
WHERE rn <= 3
ORDER BY major, rn;
```

실행 결과 ① — psql (방식 1)

```
parkyoungseo — psql -h localhost -U skala_user -d skala_db — 110x47

skala_db-> \i q23a.sql
-- Q23. 학과별 GPA 상위 3명 - Window Function (서브쿼리 방식과 CTE 방식 모두 작성)
--      ROW_NUMBER(동점 시 student_id 2차) + RANK/DENSE_RANK 비교 + COUNT() OVER
-- [방식 1] 서브쿼리 방식
SELECT student_id, name, major, gpa, rn, rnk, drnk, total_in_major
FROM (
    SELECT student_id, name, major, gpa,
           ROW_NUMBER() OVER (PARTITION BY major ORDER BY gpa DESC, student_id) AS rn,
           RANK()       OVER (PARTITION BY major ORDER BY gpa DESC)           AS rnk,
           DENSE_RANK() OVER (PARTITION BY major ORDER BY gpa DESC)           AS drnk,
           COUNT(*)      OVER (PARTITION BY major)                           AS total_in_major
    FROM student
) x
WHERE rn <= 3
ORDER BY major, rn;
 student_id |  name  | major | gpa | rn | rnk | drnk | total_in_major
-----+-----+-----+-----+----+----+-----+-----
      29 | Student_29 | BIO   | 4.90 | 1 | 1 | 1 |         196
      59 | Student_59 | BIO   | 4.90 | 2 | 1 | 1 |         196
      89 | Student_89 | BIO   | 4.90 | 3 | 1 | 1 |         196
      28 | Student_28 | CE    | 4.80 | 1 | 1 | 1 |         196
      58 | Student_58 | CE    | 4.80 | 2 | 1 | 1 |         196
      88 | Student_88 | CE    | 4.80 | 3 | 1 | 1 |         196
      25 | Student_25 | CS    | 4.50 | 1 | 1 | 1 |         196
      55 | Student_55 | CS    | 4.50 | 2 | 1 | 1 |         196
      85 | Student_85 | CS    | 4.50 | 3 | 1 | 1 |         196
      26 | Student_26 | EE    | 4.60 | 1 | 1 | 1 |         196
      56 | Student_56 | EE    | 4.60 | 2 | 1 | 1 |         196
      86 | Student_86 | EE    | 4.60 | 3 | 1 | 1 |         196
     989 | Student_989 | HR    | 4.90 | 1 | 1 | 1 |          20
     988 | Student_988 | HR    | 4.80 | 2 | 2 | 2 |          20
     987 | Student_987 | HR    | 4.70 | 3 | 3 | 3 |          20
      27 | Student_27 | ME    | 4.70 | 1 | 1 | 1 |         196
      57 | Student_57 | ME    | 4.70 | 2 | 1 | 1 |         196
      87 | Student_87 | ME    | 4.70 | 3 | 1 | 1 |         196
(18개 행)

skala_db-> □
```


SQL — 방식 2 (CTE)

```
-- [방식 2] CTE 방식
WITH ranked AS (
    SELECT student_id, name, major, gpa,
           ROW_NUMBER() OVER (PARTITION BY major ORDER BY gpa DESC, student_id) AS rn,
           RANK() OVER (PARTITION BY major ORDER BY gpa DESC) AS rnk,
           DENSE_RANK() OVER (PARTITION BY major ORDER BY gpa DESC) AS drnk,
           COUNT(*) OVER (PARTITION BY major) AS total_in_major
    FROM student
)
SELECT student_id, name, major, gpa, rn, rnk, drnk, total_in_major
FROM ranked
WHERE rn <= 3
ORDER BY major, rn;
```

실행 결과 ① — psql (방식 2 — 동일 결과)

```
parkyoungseo — psql -h localhost -U skala_user -d skala_db — 110x47
skala_db-> \i q23b.sql
-- [방식 2] CTE 방식
WITH ranked AS (
    SELECT student_id, name, major, gpa,
           ROW_NUMBER() OVER (PARTITION BY major ORDER BY gpa DESC, student_id) AS rn,
           RANK() OVER (PARTITION BY major ORDER BY gpa DESC) AS rnk,
           DENSE_RANK() OVER (PARTITION BY major ORDER BY gpa DESC) AS drnk,
           COUNT(*) OVER (PARTITION BY major) AS total_in_major
    FROM student
)
SELECT student_id, name, major, gpa, rn, rnk, drnk, total_in_major
FROM ranked
WHERE rn <= 3
ORDER BY major, rn;
 student_id | name       | major | gpa  | rn | rnk | drnk | total_in_major
-----+-----+-----+-----+----+----+-----+-----
          29 | Student_29 | BIO   | 4.90 | 1  | 1   | 1    |          196
          59 | Student_59 | BIO   | 4.90 | 2  | 1   | 1    |          196
          89 | Student_89 | BIO   | 4.90 | 3  | 1   | 1    |          196
          28 | Student_28 | CE    | 4.80 | 1  | 1   | 1    |          196
          58 | Student_58 | CE    | 4.80 | 2  | 1   | 1    |          196
          88 | Student_88 | CE    | 4.80 | 3  | 1   | 1    |          196
          25 | Student_25 | CS    | 4.50 | 1  | 1   | 1    |          196
          55 | Student_55 | CS    | 4.50 | 2  | 1   | 1    |          196
          85 | Student_85 | CS    | 4.50 | 3  | 1   | 1    |          196
          26 | Student_26 | EE    | 4.60 | 1  | 1   | 1    |          196
          56 | Student_56 | EE    | 4.60 | 2  | 1   | 1    |          196
          86 | Student_86 | EE    | 4.60 | 3  | 1   | 1    |          196
         989 | Student_989 | HR    | 4.90 | 1  | 1   | 1    |           20
         988 | Student_988 | HR    | 4.80 | 2  | 2   | 2    |           20
         987 | Student_987 | HR    | 4.70 | 3  | 3   | 3    |           20
          27 | Student_27 | ME    | 4.70 | 1  | 1   | 1    |          196
          57 | Student_57 | ME    | 4.70 | 2  | 1   | 1    |          196
          87 | Student_87 | ME    | 4.70 | 3  | 1   | 1    |          196
(18개 행)
```

```
skala_db-> □
```

실행 결과 ② — pgAdmin 4 Query Tool (두 방식 연속 실행)

Object Explorer

Servers (1)

PostgreSQL 17

PostgreSQL 18

skala_db

Databases (2)

postgres

skala_db

Costs

Catalogs

Event Triggers

Extensions

Foreign Data Wrappers

Languages

Publications

Schemas

Subscriptions

skala_db

Logic/Group Roles

Tablespaces

skala_db/skala_user@skala_db

skala_db/skala_user@skala_db

No limit

Query History

Scratch Pad X

```
1 -- Q23. 학교별 GPA 상위 3명 -> Window Function (서브쿼리 방식과 CTE 방식 모두 적절)
2 -- ROW_NUMBER(동점 시 student_id 2위) * RANK/DENSE_RANK 비고 * COUNT() OVER
3
4 -- [방식 1] 서브쿼리 방식
5 SELECT student_id, name, major, gpa, rn, rnk, drnk, total_in_major
6 FROM (
7     SELECT student_id, name, major, gpa,
8           ROW_NUMBER() OVER (PARTITION BY major ORDER BY gpa DESC, student_id) AS rn,
9           RANK() OVER (PARTITION BY major ORDER BY gpa DESC) AS rnk,
10          DENSE_RANK() OVER (PARTITION BY major ORDER BY gpa DESC) AS drnk,
11          COUNT(*) OVER (PARTITION BY major) AS total_in_major
12 FROM student
13 ) x
14 WHERE rn <= 3
15 ORDER BY major, rn;
16
17 -- [방식 2] CTE 방식
18 WITH ranked AS (
19     SELECT student_id, name, major, gpa,
20           ROW_NUMBER() OVER (PARTITION BY major ORDER BY gpa DESC, student_id) AS rn,
21           RANK() OVER (PARTITION BY major ORDER BY gpa DESC) AS rnk,
22           DENSE_RANK() OVER (PARTITION BY major ORDER BY gpa DESC) AS drnk,
23           COUNT(*) OVER (PARTITION BY major) AS total_in_major
24 FROM student
25 )
26 SELECT student_id, name, major, gpa, rn, rnk, drnk, total_in_major
27 FROM ranked
28 WHERE rn <= 3
29 ORDER BY major, rn;
30
```

Data Output

Messages

Notifications

Showing rows: 1 to 18

Page No: 1 of 1

LF Ln 30 Col 1

student_id	name	major	gpa	rn	rnk	drnk	total_in_major
PK Integer	character varying (50)	character varying (50)	numeric (3,2)	integer	integer	integer	integer
1	29	Student_29	BIO	4.90	1	1	196
2	59	Student_59	BIO	4.90	2	1	196
3	89	Student_89	BIO	4.90	3	1	196
4	28	Student_28	CE	4.80	1	1	196
5	58	Student_58	CE	4.80	2	1	196
6	88	Student_88	CE	4.80	3	1	196
7	25	Student_25	CS	4.50	1	1	196
8	55	Student_55	CS	4.50	2	1	196
9	85	Student_85	CS	4.50	3	1	196
10	26	Student_26	EE	4.60	1	1	196
11	56	Student_56	EE	4.60	2	1	196
12	86	Student_86	EE	4.60	3	1	196
13	999	Student_999	HR	4.90	1	1	20
14	988	Student_988	HR	4.80	2	2	20
15	987	Student_987	HR	4.70	3	3	20
16	27	Student_27	ME	4.30	1	1	196
17	57	Student_57	ME	4.30	2	1	196
18	87	Student_87	ME	4.30	3	1	196

Total rows: 18 Query complete 00:00:00.060

서브쿼리 방식 + CTE 방식 모두 작성 — 동일한 18행. 동점(3인 공동 1위)의 RANK·DENSE_RANK 동작은 BIO·CE·CS·EE·ME에서, 동점 없는 순차 순위(1·2·3)는 HR에서 확인. total_in_major 포함.

24. LAG()로 이전 수강 과목 대비 성적 변화 (diff·추세·score_range)

SQL

```
-- Q24. LAG()로 학생별 이전 수강 과목 대비 성적 변화 (A=4, B=3, C=2, D=1)
--      diff(현재-이전) + 상승/유지/하락 표시 + 학생별 최고-최저 차(score_range)
SELECT student_id, course, score, prev_score,
       diff,
       CASE WHEN diff > 0 THEN '상승'
            WHEN diff = 0 THEN '유지'
            WHEN diff < 0 THEN '하락'
       END AS trend,
       score_range
FROM (
  SELECT student_id, course, score,
         LAG(score) OVER (PARTITION BY student_id ORDER BY course) AS prev_score,
         score - LAG(score) OVER (PARTITION BY student_id ORDER BY course) AS diff,
         MAX(score) OVER (PARTITION BY student_id)
           - MIN(score) OVER (PARTITION BY student_id) AS score_range
  FROM (
    SELECT student_id, course,
           CASE grade WHEN 'A' THEN 4 WHEN 'B' THEN 3
                    WHEN 'C' THEN 2 WHEN 'D' THEN 1 END AS score
    FROM enroll
  ) g
) w
ORDER BY student_id, course
LIMIT 5;  -- 출력 확인용 (전체 행 수는 리포트에 별도 표기)
```

실행 결과 ① — psql

```
parkyoungseo — psql -h localhost -U skala_user -d skala_db — 110x44
skala_db-> \i q24.sql
-- Q24. LAG()로 학생별 이전 수강 과목 대비 성적 변화 (A=4, B=3, C=2, D=1)
-- diff(현재-이전) + 상승/유지/하락 표시 + 학생별 최고-최저 차(score_range)
SELECT student_id, course, score, prev_score,
       diff,
       CASE WHEN diff > 0 THEN '상승'
            WHEN diff = 0 THEN '유지'
            WHEN diff < 0 THEN '하락'
       END AS trend,
       score_range
FROM (
  SELECT student_id, course, score,
         LAG(score) OVER (PARTITION BY student_id ORDER BY course) AS prev_score,
         score - LAG(score) OVER (PARTITION BY student_id ORDER BY course) AS diff,
         MAX(score) OVER (PARTITION BY student_id)
           - MIN(score) OVER (PARTITION BY student_id) AS score_range
  FROM (
    SELECT student_id, course,
           CASE grade WHEN 'A' THEN 4 WHEN 'B' THEN 3
                WHEN 'C' THEN 2 WHEN 'D' THEN 1 END AS score
    FROM enroll
  ) g
) w
ORDER BY student_id, course
LIMIT 5; -- 출력 확인용 (전체 행 수는 리포트에 별도 표기)
```

student_id	course	score	prev_score	diff	trend	score_range
1	Course_3	2				3
1	Course_4	1	2	-1	하락	3
1	Course_5	4	1	3	상승	3
2	Course_4	1				3
2	Course_5	4	1	3	상승	3

(5개 행)

```
skala_db->
```

실행 결과 ② — pgAdmin 4 Query Tool

The screenshot shows the pgAdmin 4 Query Tool interface. The left sidebar displays the database structure, including the 'skala_db' database. The main query editor contains the same SQL query as in the first image. The 'Data Output' tab at the bottom shows the query results in a table format, matching the output from the psql terminal.

student_id	course	score	prev_score	diff	trend	score_range
1	Course_3	2				3
1	Course_4	1	2	-1	하락	3
1	Course_5	4	1	3	상승	3
2	Course_4	1				3
2	Course_5	4	1	3	상승	3

LAG + CASE — 이전 과목 점수와의 diff, 상승/유지/하락 추세, 학생별 score_range (전체 2,302행).

25. 누적 주문금액 + 3개 주문 이동평균 (ROWS BETWEEN) + 전체 합 50% 초과 첫 주문

SQL

```
-- Q25. orders를 order_id 순으로 정렬하여 누적 주문금액과 3개 주문 이동평균 계산 (ROWS BETWEEN)
--      + customer_id별 PARTITION 고객별 누적 구매금액
SELECT order_id, customer_id, amount,
       SUM(amount) OVER (ORDER BY order_id
                        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS running_total,
       ROUND(AVG(amount) OVER (ORDER BY order_id
                        ROWS BETWEEN 2 PRECEDING AND CURRENT ROW), 2) AS moving_avg_3,
       SUM(amount) OVER (PARTITION BY customer_id ORDER BY order_id
                        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS cust_running_total
FROM   orders
ORDER BY order_id
LIMIT 5;  -- 출력 확인용 (전체 3,000행)

-- Q25-2. 누적합이 전체 합의 50%를 초과하는 첫 번째 order_id
SELECT order_id, running_total, total_sum
FROM (
  SELECT order_id,
         SUM(amount) OVER (ORDER BY order_id
                          ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS running_total,
         SUM(amount) OVER () AS total_sum
  FROM orders
) t
WHERE running_total > total_sum * 0.5
ORDER BY order_id
LIMIT 1;
```

실행 결과 ① — psql (쿼리 1 — 누적합·이동평균)

```
parkyoungseo — psql -h localhost -U skala_user -d skala_db — 99x28

skala_db-> \i q25a.sql
-- Q25. orders를 order_id 순으로 정렬하여 누적 주문금액과 3개 주문 이동평균 계산 (ROWS BETWEEN)
--      + customer_id별 PARTITION 고객별 누적 구매금액
SELECT order_id, customer_id, amount,
       SUM(amount) OVER (ORDER BY order_id
                        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS running_total,
       ROUND(AVG(amount) OVER (ORDER BY order_id
                        ROWS BETWEEN 2 PRECEDING AND CURRENT ROW), 2) AS moving_avg_3,
       SUM(amount) OVER (PARTITION BY customer_id ORDER BY order_id
                        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS cust_running_total
FROM   orders
ORDER BY order_id
LIMIT 5;  -- 출력 확인용 (전체 3,000행)
 order_id | customer_id | amount | running_total | moving_avg_3 | cust_running_total
-----+-----+-----+-----+-----+-----
         1 |           2 |   18.01 |          18.01 |          18.01 |          18.01
         2 |           3 |   31.02 |          49.03 |          24.52 |          31.02
         3 |           4 |   44.03 |          93.06 |          31.02 |          44.03
         4 |           5 |   57.04 |         150.10 |          44.03 |          57.04
         5 |           6 |   70.05 |         220.15 |          57.04 |          70.05
(5개 행)

skala_db-> □
```

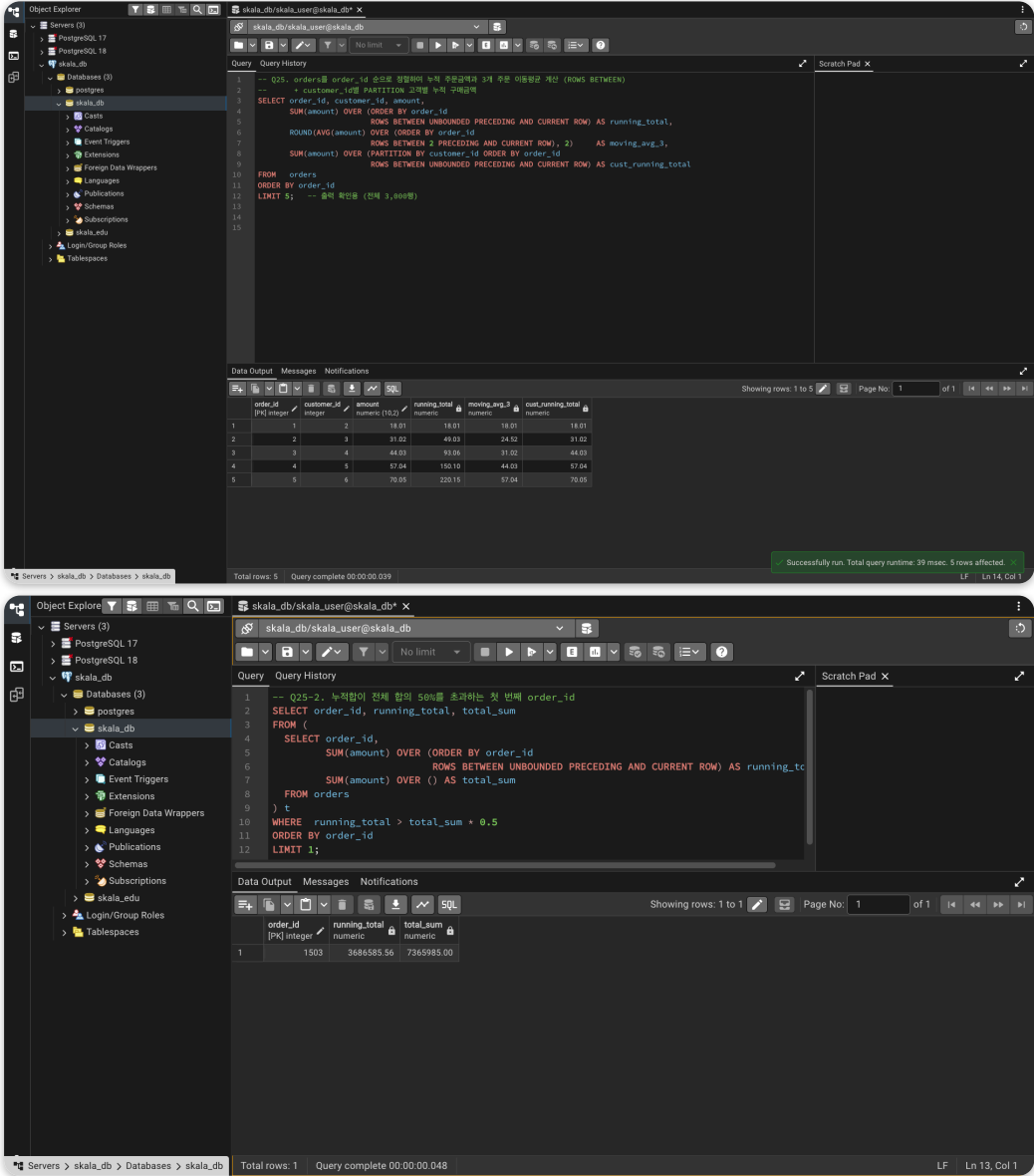
실행 결과 ① — psql (쿼리 2 — 50% 초과 첫 주문)

```
parkyoungseo — psql -h localhost -U skala_user -d skala_db — 99x27

skala_db-> \i q25b.sql
-- Q25-2. 누적합이 전체 합의 50%를 초과하는 첫 번째 order_id
SELECT order_id, running_total, total_sum
FROM (
  SELECT order_id,
         SUM(amount) OVER (ORDER BY order_id
                           ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS running_total,
         SUM(amount) OVER () AS total_sum
  FROM orders
) t
WHERE running_total > total_sum * 0.5
ORDER BY order_id
LIMIT 1;
 order_id | running_total | total_sum
-----+-----+-----
      1503 |    3686585.56 | 7365985.00
(1개 행 )

skala_db-> []
```

실행 결과 ② — pgAdmin 4 Query Tool (위: 쿼리 1 · 아래: 쿼리 2)



(a) 누적합·3개 이동평균·고객별 누적 (전체 3,000행) → (b) 누적합이 전체 합(7,365,985)의 50%를 최초로 초과하는 주문 = order_id 1503.